

Nắm Bắt Chiến Dịch, Không Phải Kết Nối: Mạng Siêu Đồ Thị Nhận Thức Phối Hợp Cho Phát Hiện xâm Nhập Đa Giai Đoạn

Anonymous Authors
Institution

Abstract

Các hệ thống phát hiện xâm nhập dựa trên nguồn gốc (provenance) phân tích nhật ký kiểm toán (audit logs) cấp hạt nhân để xác định các Mối đe dọa dai dẳng nâng cao (APT), nhưng các phương pháp hiện tại hoạt động ở mức độ chi tiết thô — phân loại toàn bộ đồ thị (KAiOS, MAGIC) hoặc các nút riêng lẻ (FLASH, ThreaTrace) — mà không quy kết việc phát hiện cho các sự kiện tấn công cụ thể. Chúng tôi giới thiệu HyperMamba, một hệ thống phát hiện các sự kiện tấn công xuyên tiến trình (crossprocess) riêng lẻ trong các luồng kiểm toán thời gian thực bằng cách duy trì trạng thái thực thể liên tục thông qua Mô hình Không gian Trạng thái Chọn lọc (Selective State Space Models). HyperMamba mô hình hóa mỗi sự kiện kiểm toán như một siêu cạnh (hyperedge) thời gian kết nối hai hoặc ba thực thể hệ thống, bảo tồn cấu trúc biến thiên số lượng đối tượng (variable-arity) nguyên bản của Mô hình Dữ liệu Chung (CDM). Một SSM Chọn lọc duy trì một vector trạng thái cho mỗi thực thể, tích lũy bối cảnh hành vi trên toàn bộ luồng; tại mỗi sự kiện, tập hợp siêu cạnh dựa trên cơ chế chú ý (attention-based hyperedge aggregation) kết hợp trạng thái của các thực thể tham gia, SSM cập nhật mỗi thực thể, và một bộ phân loại tạo ra quyết định phát hiện ở cấp độ sự kiện. Các nghiên cứu cắt bỏ (ablation studies) tiết lộ một phân cấp rõ ràng về đóng góp của các thành phần: điều biến danh tính tiến trình có cổng (gated process identity modulation) là cơ chế vượt trội cho việc tổng quát hóa chéo chiến dịch (AUPRC sự kiện giảm -55% khi bị loại bỏ), tập hợp siêu cạnh chéo thực thể mang lại đóng góp cấu trúc chính (-8.1%), và trạng thái liên tục bổ sung khả năng phân biệt còn lại cho các sự kiện tinh vi nhất (-4.6%). Việc loại bỏ cả trạng thái và danh tính tiến trình khiến kết quả phát hiện sụp đổ về gần như ngẫu nhiên (0.20 AUPRC), xác nhận rằng các đặc trưng sự kiện đơn lẻ không thể phân biệt được các cuộc tấn công xuyên tiến trình. Trên tập dữ liệu DARPA TC Engagement 3, HyperMamba đạt được AUPRC cấp sự kiện là 0.76 trên

Theia (nhân xuyên tiến trình, đánh giá chéo chiến dịch) và 0.97 trên CADETS (nhân rộng, đánh giá chéo chiến dịch), với $FPR \leq 0.0011$, trong khi xử lý các sự kiện với tốc độ gấp $194\times$ tốc độ thu thập dữ liệu kiểm toán đỉnh điểm khi suy luận trên một GPU duy nhất.

1 Giới thiệu

Các Hệ thống Phát hiện Xâm nhập Mạng (NIDS) dựa trên nguồn gốc phân tích nhật ký kiểm toán cấp hạt nhân để phát hiện các Mối đe dọa dai dẳng nâng cao (APT) trên các máy chủ được giám sát. Các hệ thống tiên tiến gần đây — KAIROS [3], MAGIC [7], FLASH [8], và ThreaTrace [9] — đã thể hiện khả năng phát hiện mạnh mẽ. Tuy nhiên, các hệ thống này hoạt động ở mức độ chi tiết về cơ bản là thô: KAIROS và MAGIC gắn điểm dị thường cho toàn bộ đồ thị nguồn gốc, trong khi FLASH và ThreaTrace phân loại các nút (tiến trình hoặc tệp) riêng lẻ. Không có hệ thống nào đưa ra phân loại ở cấp độ sự kiện (event-level). Khi một nhà phân tích báo nhận được cảnh báo cấp nút đánh dấu một tiến trình là độc hại, nhà phân tích vẫn phải kiểm tra thủ công hàng ngàn sự kiện của tiến trình đó để xác định hoạt động cụ thể nào cấu thành cuộc tấn công. Khoảng trống pháp y giữa phát hiện và quy kết này làm hạn chế tiện ích vận hành của các hệ thống hiện tại.

Khoảng cách về độ chi tiết không chỉ đơn thuần là bất tiện — nó phản ánh một hạn chế mang tính cấu trúc. Các sự kiện tấn công xuyên tiến trình (crossprocess attack events) — một nhân mà chúng tôi định nghĩa trong Mục 3.4 để chỉ các sự kiện do các tiến trình con được sinh ra trong các chiến dịch APT đa giai đoạn thực thi, ngoại trừ tiến trình điểm vào (entry-point) ban đầu — sử dụng cùng các lệnh gọi hệ thống như các hoạt động nền lành tính. Phân phối đặc trưng trên mỗi sự kiện của chúng thỏa mãn $P(\mathbf{x}_i \mid \text{tấn công}) \approx P(\mathbf{x}_i \mid \text{lành tính})$, khiến chúng không thể phân biệt được về mặt thống kê nếu xét riêng lẻ. Một hệ thống đánh giá các sự kiện

một cách độc lập không thể phân tách một cách đáng tin cậy các hoạt động đọc và ghi độc hại khỏi các hoạt động lành tính. Tín hiệu phân biệt chỉ tồn tại trong lịch sử hành vi tích lũy (accumulated behavioral history) của thực thể thực thi: chuỗi các hoạt động trước đó tiết lộ vai trò của thực thể trong một chiến dịch đang diễn ra. Các hệ thống hiện tại loại bỏ tín hiệu này bằng cách hoạt động trên các ảnh chụp nhanh (snapshots) đồ thị tĩnh mà không có trạng thái thực thể liên tục.

Chúng tôi giới thiệu HyperMamba, một NIDS dựa trên nguồn gốc giúp phát hiện các sự kiện tấn công xuyên tiến trình riêng lẻ trong các luồng kiểm toán thời gian thực bằng cách duy trì trạng thái thực thể liên tục thông qua Mô hình Không gian Trạng thái Chọn lọc (Selective State Space Models). HyperMamba mô hình hóa mỗi sự kiện kiểm toán như một siêu cạnh thời gian kết nối hai hoặc ba thực thể hệ thống, bảo tồn cấu trúc biến thiên số lượng đối tượng nguyên bản của Mô hình Dữ liệu Chung (CDM). Một SSM Chọn lọc duy trì và cập nhật một vectơ trạng thái cho mỗi thực thể trong hệ thống, tích lũy bối cảnh hành vi trên toàn bộ luồng sự kiện. Tại mỗi sự kiện, mô hình tập hợp trạng thái của các thực thể tham gia thông qua cơ chế tập hợp siêu cạnh dựa trên chú ý, cập nhật trạng thái của từng thực thể thông qua vòng lặp SSM và phân loại sự kiện bằng cách sử dụng cả trạng thái được cập nhật và các đặc trưng của chính sự kiện đó. Kết quả là khả năng phát hiện thời gian thực, cấp độ sự kiện với quy kết pháp y cho từng sự kiện.

Các đóng góp của chúng tôi bao gồm:

1. Hình thức hóa siêu đồ thị nguồn gốc (C1). Chúng tôi hình thức hóa các luồng kiểm toán thành các siêu đồ thị thời gian với siêu cạnh có kích thước biến thiên và chứng minh thông qua thực nghiệm cắt bỏ có kiểm soát rằng việc tập hợp trạng thái xuyên thực thể trong các siêu cạnh mang lại sự cải thiện 8.1% AUPRC so với việc theo dõi riêng lẻ từng thực thể.
2. Danh tính tiến trình ngữ nghĩa và trạng thái thời gian như các cơ chế phát hiện hỗ trợ cho nhau (C2). Chúng tôi chứng minh bằng thực nghiệm rằng sự kết hợp của danh tính tiến trình ngữ nghĩa và trạng thái thực thể thời gian là cần thiết đồng thời để phát hiện tấn công xuyên tiến trình. Việc loại bỏ cả hai thành phần làm sụp đổ AUPRC sự kiện từ 0.76 xuống 0.20 trên đánh giá chéo chiến dịch. Danh tính tiến trình là thành phần đóng góp đơn lẻ vượt trội (AUPRC giảm -55% khi bị loại bỏ), trong khi trạng thái tích lũy cung cấp khả năng phân biệt còn lại cho các sự kiện tinh vi nhất (-4.6%). Không thành phần nào đứng đơn lẻ là đủ (Mục 5.3).
3. Danh tính tiến trình có cổng cho việc tổng quát hóa chéo chiến dịch (C3). Chúng tôi giới thiệu một cơ

chế điều biến đường dẫn tiến trình có cổng nhằm cung cấp các điểm neo ngữ nghĩa bất biến theo chiến dịch. Loại bỏ danh tính tiến trình trong khi vẫn giữ nguyên kiến trúc trạng thái đầy đủ làm giảm AUPRC chéo chiến dịch từ 0.76 xuống 0.34 (Mục 5.3).

4. Phát hiện cấp sự kiện ở quy mô vận hành (C4). HyperMamba đạt AUPRC cấp sự kiện là 0.76 (chéo chiến dịch) với FPR là 0.0011, trong khi xử lý dữ liệu với tốc độ gấp $194\times$ tốc độ thu thập dữ liệu kiểm toán đỉnh điểm khi suy luận trên một GPU duy nhất (Mục 5.2).

Mục 2 giới thiệu mô hình dữ liệu nguồn gốc và kiến trúc nền tảng về SSM. Mục 3 định nghĩa mô hình mối đe dọa và bài toán phát hiện. Mục 4 trình bày chi tiết kiến trúc HyperMamba. Mục 5 trình bày đánh giá. Mục 7 và 6 thảo luận về các nghiên cứu liên quan và các giới hạn.

2 Kiến thức Nền tảng

Mục này định nghĩa ba khái niệm cần thiết để theo dõi kiến trúc (Mục 4): mô hình dữ liệu nguồn gốc, sự hình thức hóa của nó dưới dạng một siêu đồ thị thời gian, và vòng lặp của Mô hình Không gian Trạng thái Chọn lọc.

2.1 Mô hình Dữ liệu Nguồn gốc

Các hệ thống kiểm toán cấp máy chủ hiện đại dựa trên Mô hình Dữ liệu Chung (CDM) của DARPA ghi lại mọi hoạt động qua trung gian hạt nhân dưới dạng một sự kiện có kiểu kết nối một chủ thể (subject - tiến trình thực thi) với một hoặc hai đối tượng (objects - mục tiêu của hoạt động). Một thao tác đọc tệp tạo ra một sự kiện với một chủ thể và một đối tượng (`process` $\rightarrow$ `file`); một thao tác ánh xạ bộ nhớ của một thư viện chia sẻ tạo ra một sự kiện với một chủ thể và hai đối tượng (`process` $\rightarrow$ `file` $\rightarrow$ `memory_object`). Mỗi sự kiện mang siêu dữ liệu: một loại sự kiện rời rạc (một trong ~ 40 loại sự kiện CDM như `EVENT_READ`, `EVENT_MMAP`), các thuộc tính liên tục (số byte, cờ), và dấu thời gian với độ phân giải nano giây. Trong một chiến dịch kéo dài nhiều ngày, một máy chủ duy nhất có thể tạo ra hàng chục triệu sự kiện như vậy.

2.2 Siêu đồ thị Nguồn gốc

Chúng tôi hình thức hóa luồng kiểm toán dưới dạng một siêu đồ thị thời gian.

Definition 1 (Siêu đồ thị Nguồn gốc). Một siêu đồ thị nguồn gốc là một bộ $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, trong đó $\mathcal{V}$ là tập hữu hạn các thực thể hệ thống (tiến trình, tệp, socket,

đối tượng bộ nhớ) và $\mathcal{E} = \{e_1, e_2, \dots\}$ là chuỗi các siêu cạnh được sắp xếp theo trình tự thời gian. Mỗi siêu cạnh $e_t = (v_{\text{subj}}, v_{\text{obj}}, v_{\text{obj2}}, \tau_t, \mathbf{a}_t)$ kết nối $|V_{e_t}| \in \{2, 3\}$ thực thể, trong đó v_{obj2} vắng mặt đối với các sự kiện kích thước 2, τ_t là dấu thời gian của sự kiện, và $\mathbf{a}_t$ là vectơ thuộc tính.

Ví dụ, khi tiến trình Firefox bị xâm phạm ánh xạ tệp nhị phân độc hại `malicious.so` vào bộ nhớ có thể thực thi, siêu cạnh `EVENT_MMAP` được tạo ra kết nối ba thực thể: Firefox (v_{subj}), tệp `malicious.so` (v_{obj}), và vùng bộ nhớ được cấp phát (v_{obj2}).

Tại sao lại là siêu cạnh, không phải cạnh cặp đôi. Các NIDS dựa trên nguồn gốc trước đây [3, 7–9] phân rã mỗi sự kiện đa đối tượng thành các cạnh cặp đôi độc lập, loại bỏ cấu trúc đồng xuất hiện. Trong tập dữ liệu DARPA TC E3 Theia, 2.8% các sự kiện là siêu cạnh kích thước 3. Mặc dù tỷ lệ này có vẻ nhỏ, các sự kiện kích thước 3 lại nhiều gấp 3.0× trong các sự kiện tấn công xuyên tiến trình so với các sự kiện lành tính, được thúc đẩy chủ yếu bởi các hoạt động `EVENT_MMAP` (ví dụ: một tiến trình bị xâm phạm ánh xạ một thư viện chia sẻ độc hại vào bộ nhớ thực thi). Sự làm giàu này là động lực để duy trì kích thước siêu cạnh nguyên bản thay vì phân rã thành các tương tác cặp đôi. Hình 1 minh họa điều này với một ví dụ cụ thể từ chiến dịch tấn công Theia.

2.3 Mô hình Không gian Trạng thái Chọn lọc

Mô hình Không gian Trạng thái (SSMs) định nghĩa một phép đệ quy tuyến tính trên một trạng thái ẩn $\mathbf{h} \in \mathbb{R}^n$, ánh xạ một chuỗi đầu vào $x(t)$ tới đầu ra $y(t)$ thông qua động lực học thời gian liên tục:

$$\mathbf{h}'(t) = \mathbf{A}\mathbf{h}(t) + \mathbf{B}x(t), \quad y(t) = \mathbf{C}\mathbf{h}(t) \quad (1)$$

trong đó $\mathbf{A} \in \mathbb{R}^{n \times n}$ quản lý sự suy giảm trạng thái và $\mathbf{B} \in \mathbb{R}^{n \times 1}$ kiểm soát việc tiêm đầu vào. Đối với các chuỗi rời rạc, động lực học liên tục được rời rạc hóa với bước nhảy Δ để mang lại:

$$\bar{\mathbf{A}} = \exp(\Delta\mathbf{A}), \quad \bar{\mathbf{B}} = (\Delta\mathbf{A})^{-1}(\bar{\mathbf{A}} - \mathbf{I}) \cdot \Delta\mathbf{B} \quad (2)$$

$$\mathbf{h}_t = \bar{\mathbf{A}}\mathbf{h}_{t-1} + \bar{\mathbf{B}}x_t, \quad y_t = \mathbf{C}\mathbf{h}_t \quad (3)$$

SSM Chọn lọc [5] khiến $\mathbf{B}$, $\mathbf{C}$, và Δ phụ thuộc vào đầu vào, cho phép mô hình chọn lọc giữ lại hoặc loại bỏ thông tin tại mỗi bước dựa trên đầu vào hiện tại. Trong Mục 4.5, chúng tôi điều chỉnh cơ chế này để hoạt động trên các trạng thái thực thể bên trong một siêu đồ thị, nơi Δ được điều kiện hóa thêm dựa trên thời gian trôi qua kể từ lần gần nhất thực thể được quan sát.

3 Mô hình Mối đe dọa

3.1 Mô hình Kẻ tấn công

Chúng tôi xem xét một kẻ tấn công thuộc nhóm Mối đe dọa dai dẳng nâng cao (APT) đang thực thi các chiến dịch tấn công xuyên tiến trình đa giai đoạn trên một máy chủ được giám sát.

Kiến thức. Kẻ tấn công biết rằng tính năng ghi nhật ký kiểm toán cấp hạt nhân đang hoạt động trên máy chủ đích. Trong đánh giá chính, kẻ tấn công không có kiến thức về kiến trúc hoặc các tham số của mô hình phát hiện (cài đặt hộp đen). Chúng tôi thảo luận về các tác động của một kẻ tấn công hộp trắng nhằm mục tiêu vào cơ chế truyền trạng thái SSM trong Mục 6.”

Khả năng. Kẻ tấn công có thể thực thi các hoạt động tùy ý thông qua các API hệ thống tiêu chuẩn: sinh ra các tiến trình, đọc và ghi tệp, ánh xạ các vùng bộ nhớ và thiết lập kết nối mạng. Tất cả các hoạt động như vậy đều được cơ sở hạ tầng ghi nhật ký của hạt nhân ghi lại dưới dạng các sự kiện kiểm toán. Kẻ tấn công không thể can thiệp hoặc chặn nhật ký kiểm toán cấp hạt nhân. Giả định này là tiêu chuẩn trong các tài liệu NIDS dựa trên nguồn gốc và phản ánh các hệ thống triển khai nơi việc thu thập kiểm toán chạy ở mức đặc quyền cao hơn các tiến trình không gian người dùng.

Mục tiêu. Mục tiêu của kẻ tấn công là thực thi một chiến dịch xuyên tiến trình đa giai đoạn hoàn chỉnh — khai thác ban đầu một ứng dụng điểm vào, triển khai các công cụ thứ cấp thông qua các tiến trình con, thiết lập các kênh điều khiển và chỉ huy (C2), và rò rỉ dữ liệu — trong khi trốn tránh sự phát hiện ở cấp độ sự kiện. Kẻ tấn công đạt được sự trốn tránh nếu các sự kiện hệ thống riêng lẻ do các tiến trình con của chiến dịch tạo ra không bị hệ thống phát hiện đánh dấu là độc hại.

3.2 Sự kiện Tấn công DARPA TC E3

Chúng tôi dựa trên mô hình mối đe dọa vào một chiến dịch đa giai đoạn cụ thể từ tập dữ liệu DARPA TC Engagement 3 Theia. Cuộc tấn công diễn ra qua bốn giai đoạn riêng biệt, mỗi giai đoạn tạo ra các sự kiện kiểm toán trong nhật ký cấp hạt nhân của máy chủ:

1. Xâm phạm ban đầu. Kẻ tấn công khai thác một lỗ hổng trong tiến trình duyệt Firefox. Firefox thực thi một `EVENT_WRITE` để thả một mã nhị phân độc hại tại `/home/admin/clean`. Sự kiện đơn lẻ này là điểm vào (entry point) — Firefox là tiến trình bị xâm phạm.
2. Kích hoạt tải trọng. Firefox sinh ra tải trọng đã được thả, tải trọng này sau đó sẽ khởi chạy các tiến trình hạ nguồn bao gồm `pulseaudio` và `gconf-helper` thông qua `EVENT_EXECUTE`. Các tiến

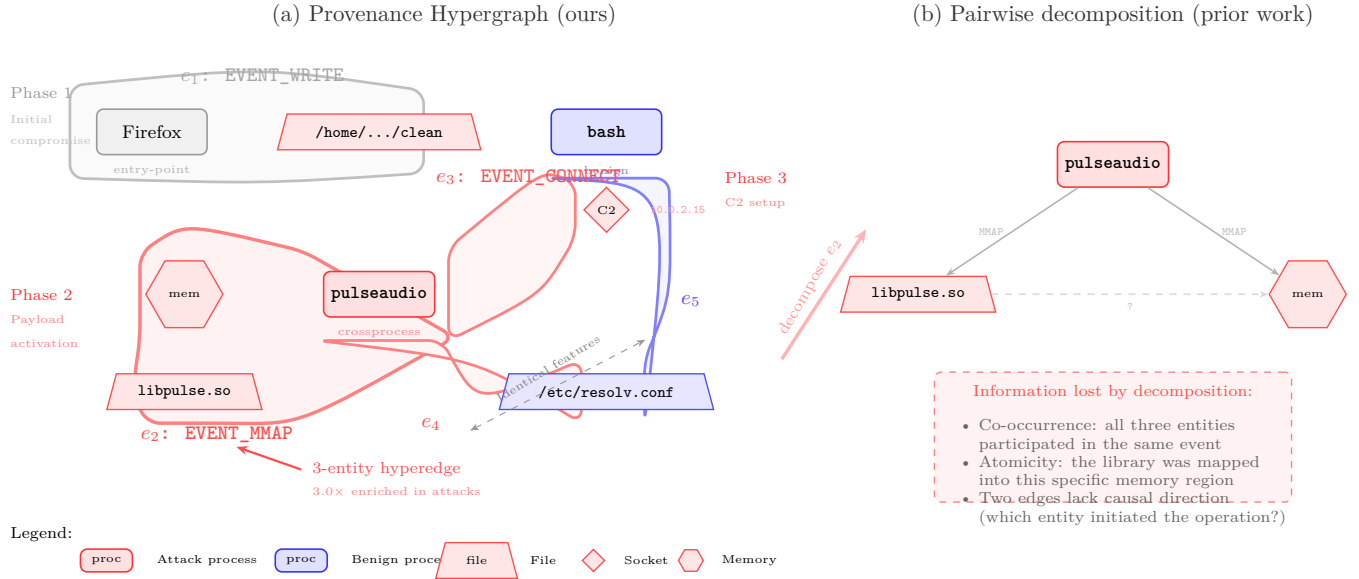


Figure 1: Provenance hypergraph from the DARPA TC E3 Theia attack campaign. (a) Each audit event is a temporal hyperedge connecting 2–3 system entities. The `EVENT_MMAP` hyperedge e_2 natively connects three entities (process, file, memory region) as a single atomic operation. Events e_4 (attack) and e_5 (benign) have identical per-event features—both read `/etc/resolv.conf` with the same byte count—but are distinguishable through the accumulated state of their subject entities. Gray shading marks the entry-point event (excluded from the crossprocess label). (b) Prior systems decompose multi-object events into independent pairwise edges, discarding co-occurrence structure and causal atomicity. Size-3 events are $3.0\times$ enriched among crossprocess attack events.

trình con này tải các thư viện chia sẻ (`EVENT_MMAP`) và đọc các tệp cấu hình hệ thống (`EVENT_READ` trên `/etc/resolv.conf`, `/etc/hosts`).

- Điều khiển và Chỉ huy. Các tiến trình được sinh ra thiết lập các kết nối reverse shell đến cơ sở hạ tầng của kẻ tấn công thông qua `EVENT_CONNECT` đến một địa chỉ IP bên ngoài. Các sự kiện `EVENT_SENDTO` và `EVENT_RECVFROM` tiếp theo mang các lệnh và phản hồi C2.
- Rò rỉ dữ liệu. Dưới sự điều hướng của C2, các tiến trình con đọc các tệp nhạy cảm (`EVENT_READ`) và truyền nội dung của chúng tới máy chủ của kẻ tấn công (`EVENT_SENDTO`). Các tiến trình bổ sung có thể được sinh ra để thực hiện việc trinh sát mạng ngang hàng.

Định nghĩa nhân xuyên tiến trình. Trong bài báo này, nhân xuyên tiến trình (crossprocess) đề cập đến các sự kiện được thực thi bởi các tiến trình con được sinh ra ở hạ nguồn của tiến trình điểm vào — trong tập dữ liệu Theia, đây là các trường thể của `pulseaudio` và `gconf-helper` và bất kỳ tiến trình nào mà chúng sinh ra sau đó. Bản thân tiến trình điểm vào (Firefox) bị loại trừ khỏi nhân này. Việc loại trừ này là có chủ ý: việc khai thác ban đầu đối với Firefox có thể tạo ra các cuộc

gọi hệ thống bất thường (ví dụ: trình duyệt ghi một tệp nhị phân thực thi), có thể phát hiện được bằng các phương pháp không trạng thái đơn giản hơn. Bài toán phát hiện khó hơn — và là mục tiêu của HyperMamba — là xác định các tiến trình con ở giai đoạn sau, các sự kiện riêng lẻ của chúng về mặt vận hành không thể phân biệt được với hoạt động hệ thống lành tính.

3.3 Tại sao các cuộc tấn công xuyên tiến trình lại khó đối với việc phát hiện không trạng thái

Các cuộc tấn công xuyên tiến trình được thực thi bởi các tiến trình con mà tiến trình điểm vào bị xâm phạm sinh ra để thực hiện các giai đoạn hoạt động của chiến dịch — trinh sát, di chuyển ngang, tích lũy dữ liệu và rò rỉ. Các tiến trình con này (ví dụ: các phiên bản `pulseaudio` được sinh ra bởi tải trọng được thả, hoặc các tập lệnh shell được tiêm vào) sử dụng chính xác các lệnh gọi hệ thống giống như các tiến trình nền lành tính: `EVENT_READ` trên các tệp cấu hình, `EVENT_WRITE` vào các thư mục tạm thời, `EVENT_MMAP` để tải thư viện chia sẻ, và `EVENT_CONNECT` đến các máy chủ bên ngoài. Một bộ phân loại không trạng thái, đánh giá mỗi sự kiện một cách riêng lẻ, quan sát các lệnh gọi này mà không có bối cảnh lịch sử và đối mặt với một sự mơ hồ

cơ bản.

Ví dụ cụ thể. Hãy xem xét hai sự kiện được rút ra từ tập dữ liệu DARPA TC E3 Theia:

1. `bash` (lành tính) thực thi `EVENT_READ` trên `/etc/resolv.conf`, truyền 242 byte, $\Delta t = 0.4$ giây kể từ sự kiện trước đó của nó.
2. `pulseaudio` (tần công xuyên tiến trình) thực thi `EVENT_READ` trên `/etc/resolv.conf`, truyền 242 byte, $\Delta t = 0.3$ giây kể từ sự kiện trước đó của nó.

Cả hai sự kiện chia sẻ cùng loại sự kiện, cùng tệp đích, và các đặc trưng liên tục gần như giống hệt nhau (số byte, cờ, thời gian giữa hai sự kiện liên tiếp). Một vectơ đặc trưng được trích xuất từ bất kỳ sự kiện nào một cách riêng lẻ đều không thể phân biệt được về mặt thống kê. Một cách chính thức hơn, các phân phối đặc trưng trên mỗi sự kiện thỏa mãn $P(\mathbf{x}_i \mid \text{tần công}) \approx P(\mathbf{x}_i \mid \text{lành tính})$ đối với phần lớn các sự kiện tấn công xuyên tiến trình. Tín hiệu phân biệt không nằm ở bản thân sự kiện, mà nằm ở lịch sử hành vi tích lũy của thực thể chủ thể: `pulseaudio` được sinh ra bởi một chuỗi tiến trình bị xâm phạm bắt nguồn từ Firefox, trước đó đã thiết lập kết nối C2 và đã ghi vào các thư mục tích lũy dữ liệu — một mô hình chỉ có thể nhìn thấy thông qua trạng thái liên tục.

Kiểm chứng thực nghiệm. Nghiên cứu cắt bỏ trong Mục 5.3 cung cấp bằng chứng thực nghiệm trực tiếp cho điều kiện lẫn tránh này. Việc loại bỏ cả trạng thái thực thể tích lũy và danh tính tiến trình ngữ nghĩa khỏi HyperMamba (trong khi vẫn giữ nguyên bộ mã hóa sự kiện, bộ tập hợp siêu cạnh và bộ phân loại) khiến AUPRC kiểm tra chéo chiến dịch sụp đổ từ 0.76 xuống 0.20 — gần như ngẫu nhiên. Các sự kiện tấn công xuyên tiến trình, theo thiết kế, không thể xác định được từ các đặc trưng sự kiện đơn lẻ (loại sự kiện và các đại lượng liên tục) nếu chỉ xét riêng lẻ.

3.4 Bài toán Phát hiện

Cho một luồng sự kiện kiểm toán liên tục theo trình tự thời gian $e_1, e_2, \dots$ được tạo ra bởi một máy chủ giám sát duy nhất, bài toán phát hiện là phân loại mỗi sự kiện e_t là lành tính ($y_t = 0$) hoặc tấn công xuyên tiến trình ($y_t = 1$) tại thời điểm sự kiện đến. Về mặt hình thức, bộ phát hiện là một hàm:

$$f_\theta(e_t \mid e_1, \dots, e_{t-1}) \rightarrow [0, 1] \quad (4)$$

tạo ra xác suất $\hat{y}_t$ sử dụng sự kiện hiện tại e_t làm đầu vào và tất cả các sự kiện đã quan sát trước đó $e_1, \dots, e_{t-1}$ làm bối cảnh lịch sử. Bộ phát hiện không thể truy cập các sự kiện tương lai $e_{t+1}, e_{t+2}, \dots$ — việc phân loại phải mang tính nhân quả nghiêm ngặt.

Phạm vi. Nghiên cứu này giải quyết việc phát hiện dựa trên máy chủ từ nhật ký kiểm toán cấp hạt nhân. Phát hiện xâm nhập dựa trên luồng mạng, kiểm tra tải trọng, và khớp chữ ký nằm ngoài phạm vi của bài báo này.

4 Kiến trúc HyperMamba

HyperMamba xử lý các sự kiện kiểm toán theo trình tự thời gian nghiêm ngặt dưới dạng một luồng liên tục. Mỗi sự kiện là một siêu cạnh thời gian kết nối hai hoặc ba thực thể hệ thống (Mục 2.2). Đối với mỗi sự kiện đến, mô hình thực hiện một lượt truyền thẳng (forward pass) năm giai đoạn để đọc và cập nhật các trạng thái thực thể liên tục. Hình 2 tóm tắt luồng xử lý; các Mục 4.2–4.7 trình bày chi tiết từng giai đoạn.

4.1 Tổng quan

Gọi e_t biểu thị sự kiện kiểm toán đến tại bước rời rạc t , với các thực thể tham gia $V_{e_t} \subseteq \mathcal{V}$, trong đó $|V_{e_t}| \in \{2, 3\}$. Chúng tôi ký hiệu ba khe thực thể là v_{subj} (tiền trình chủ thể), v_{obj} (đối tượng chính), và v_{obj2} (đối tượng phụ, vắng mặt với sự kiện kích thước 2). Mỗi thực thể $v \in \mathcal{V}$ duy trì một vectơ trạng thái liên tục $\mathbf{s}_v \in \mathbb{R}^d$ trong `EntityStateBank`, với $d = 256$ trong suốt bài báo này.

HyperMamba xử lý e_t theo năm giai đoạn:

1. Mã hóa Sự kiện (Mục 4.2). Loại sự kiện, các đặc trưng liên tục (số byte, cờ, thống kê thời gian đến liên tiếp), và đường dẫn tiến trình của chủ thể được chiếu và kết hợp thành biểu diễn sự kiện dày đặc $\mathbf{f}_e \in \mathbb{R}^{2d}$. Danh tính đường dẫn tiến trình được tiềm thông qua một cổng thẳng dư cộng (gated additive residual) với dropout danh tính ở giai đoạn đào tạo để ngăn chặn việc ghi nhớ theo từng tiến trình.
2. Truy xuất Trạng thái Thực thể (Mục 4.3). Đối với mỗi thực thể $v \in V_{e_t}$, trạng thái hiện tại $\mathbf{s}_v$ và thời gian đã trôi qua Δt_v kể từ lần gần nhất v được quan sát được đọc từ `EntityStateBank`. Một mặt nạ hợp lệ m_v đánh dấu các đối tượng phụ vắng mặt.
3. Tập hợp Siêu cạnh, $\mathcal{V} \rightarrow \mathcal{E}$ (Mục 4.4). Các trạng thái thực thể được truy xuất được tập hợp thành một biểu diễn siêu cạnh duy nhất $\mathbf{x}_e \in \mathbb{R}^d$ thông qua cơ chế chú ý tích vô hướng có chia tỷ lệ, nơi các đặc trưng sự kiện $\mathbf{f}_e$ đóng vai trò là truy vấn và các trạng thái thực thể — được nối với các nhúng vai trò học được $\mathbf{r}_v$ và thời gian trôi qua theo tỷ lệ logarit $\log(1 + \Delta t_v)$ — đóng vai trò là khóa và giá trị.

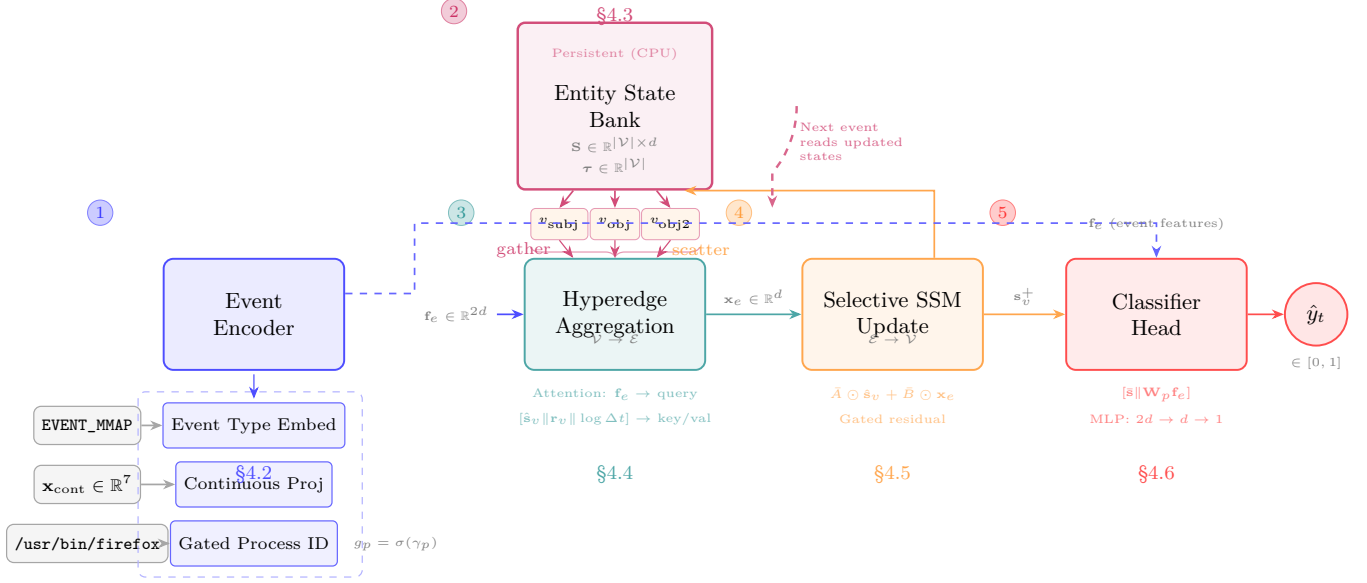


Figure 2: HyperMamba architecture. Each audit event e_t is processed in five stages. (1) The event encoder combines event type, continuous features, and gated process identity into $\mathbf{f}_e$. (2) Entity states $\hat{\mathbf{s}}_v$ and elapsed times Δt_v are gathered from the persistent **EntityStateBank**. (3) Attention-based hyperedge aggregation ($\mathcal{V} \rightarrow \mathcal{E}$) fuses entity states conditioned on $\mathbf{f}_e$. (4) A Selective SSM updates each entity’s state ($\mathcal{E} \rightarrow \mathcal{V}$) and scatters results back to the bank. (5) Post-update states and event features are concatenated and classified. The bank persists across events, creating a recurrent information pathway across entities and time.

4. Cập nhật Trạng thái SSM, $\mathcal{E} \rightarrow \mathcal{V}$ (Mục 4.5). Trạng thái của mỗi thực thể tham gia $\mathbf{s}_v$ được cập nhật thông qua Mô hình Không gian Trạng thái Chọn lọc. Cập nhật này đặc thù theo vai trò (chủ thể và đối tượng nhận cập nhật bất đối xứng), nhận biết thời gian (bước rời rạc hóa Δ_i tích hợp Δt_v), và có cổng (một cổng thẳng dư học được kiểm soát mức độ đầu ra SSM thay thế trạng thái trước đó).
5. Phân loại (Mục 4.6). Các trạng thái thực thể sau khi cập nhật được lấy trung bình gộp (mean-pooled, che bằng mặt nạ hợp lệ) và nối với $\mathbf{f}_e$ để tạo ra một logit vô hướng $\hat{y}_t = \sigma(\text{MLP}([\bar{\mathbf{s}} \parallel \mathbf{W}_p \mathbf{f}_e]))$, trong đó $\hat{y}_t \in [0, 1]$ là xác suất e_t là một sự kiện tấn công xuyên tiến trình.

Trạng thái được ghi vào ngân hàng ở giai đoạn 4 cho thực thể v là trạng thái được đọc ở giai đoạn 2 lần tiếp theo v tham gia vào bất kỳ sự kiện nào. Điều này tạo ra một đường dẫn thông tin hồi quy giữa các thực thể và qua thời gian, với bộ nhớ giới hạn ở $\mathcal{O}(|\mathcal{V}| \times d)$ không phụ thuộc vào số lượng sự kiện đã xử lý. Việc đào tạo sử dụng Lan truyền ngược qua thời gian bị cản trở bởi sự tách rời tại các ranh giới chunk (Mục 4.7).

4.2 Mã hóa Sự kiện và Điều biến Danh tính Tiến trình

Mỗi sự kiện kiểm toán e_t mang ba loại đầu vào thô: một loại sự kiện rời rạc (một trong ~ 40 loại sự kiện CDM như `EVENT_READ`, `EVENT_MMAP`, `EVENT_CONNECT`), một vectơ đặc trưng liên tục $\mathbf{x}_{\text{cont}} \in \mathbb{R}^{n_c}$ (số byte, cờ, thống kê thời gian đến liên tiếp), và đường dẫn hệ thống tệp của tiến trình chủ thể (ví dụ: `/usr/bin/bash`, `/home/admin/clean`). Bộ mã hóa sự kiện chiếu các đặc trưng này vào một biểu diễn dày đặc $\mathbf{f}_e \in \mathbb{R}^{2d}$ dùng làm đầu vào cho tất cả các giai đoạn tiếp theo.

Loại sự kiện và đặc trưng liên tục. Loại sự kiện được ánh xạ tới một nhúng học được $\mathbf{e}_{\text{type}} = \text{Embed}_{\text{type}}(\text{type}(e_t)) \in \mathbb{R}^d$. Các đặc trưng liên tục được chiếu thông qua một lớp tuyến tính $\mathbf{c} = \mathbf{W}_c \mathbf{x}_{\text{cont}} + \mathbf{b}_c \in \mathbb{R}^d$.

Điều biến danh tính tiến trình có cổng. Các định danh thực thể số nguyên được gán trong quá trình tiền xử lý không có ý nghĩa ngữ nghĩa: cùng một tệp nhị phân độc hại `/home/admin/clean` nhận các định danh nguyên khác nhau trong các chiến dịch khác nhau vì từ vựng thực thể được xây dựng trên từng tập dữ liệu. Để cung cấp một tín hiệu ngữ nghĩa có thể tổng quát hóa trên các chiến dịch, chúng tôi nhúng đường dẫn tiến trình của chủ thể thông qua phép tra cứu học được $\mathbf{p}_{\text{raw}} = \text{Embed}_{\text{proc}}(\text{path}(v_{\text{subj}})) \in \mathbb{R}^{64}$ từ một từ vựng về

tên tiến trình đã biết (được định nghĩa trong Mục 5.1). Những này được chiếu tới chiều mô hình $\mathbf{p} = \mathbf{W}_p \mathbf{p}_{\text{raw}} \in \mathbb{R}^d$.

Danh tính tiến trình được thêm dưới dạng một cổng thẳng dư cộng (gated additive residual) vào những loại sự kiện thay vì thông qua phép nối:

$$\mathbf{e}_{\text{mod}} = \mathbf{e}_{\text{type}} + g_p \cdot \mathbf{p} \quad (5)$$

trong đó $g_p = \sigma(\gamma_p)$ is a scalar gate controlled by a single learnable parameter γ_p , initialized to -2.0 so that $g_p \approx 0.12$ at the start of training. Danh tính tiến trình được thêm vào những loại sự kiện thay vì các đặc trưng liên tục vì đường dẫn tiến trình điều biến ngữ nghĩa sự kiện (ví dụ: một `EVENT_MMAP` bởi `firefox` mang ý nghĩa ngữ cảnh khác với một hoạt động tương tự bởi `pulseaudio`), trong khi các đặc trưng liên tục đo lường các đại lượng không phụ thuộc vào tiến trình như số byte. Khởi tạo cổng gần bằng không buộc mô hình trước tiên phải học từ loại sự kiện và các đặc trưng liên tục trước khi tín hiệu danh tính tiến trình được trộn vào. Nếu không có cổng, tín hiệu tiến trình cộng vào sẽ không bị ràng buộc tại thời điểm khởi tạo, tạo ra một lỗi tắt gradient qua danh tính tiến trình làm bỏ qua việc phân biệt ở cấp độ sự kiện. Sự cần thiết của danh tính tiến trình được xác nhận bằng thực nghiệm trong Mục 5.3: loại bỏ những tiến trình trong khi vẫn giữ nguyên kiến trúc trạng thái đầy đủ làm giảm AUPRC chéo chiến dịch từ 0.76 xuống 0.34.

Dropout danh tính. Trong quá trình đào tạo, danh tính tiến trình của mỗi sự kiện được thay thế độc lập bằng một mã thông báo không xác định (chỉ mục 0) với xác suất $\rho = 0.3$:

$$\text{path}(v_{\text{subj}}) \leftarrow \begin{cases} \text{path}(v_{\text{subj}}) & \text{với xác suất } 1 - \rho \\ \text{<unk>} & \text{với xác suất } \rho \end{cases} \quad (6)$$

Điều này ngăn bộ phân loại phụ thuộc vào danh tính tiến trình như một tín hiệu phát hiện đầy đủ. Mô hình phải duy trì một đường dẫn phát hiện hoạt động hiệu quả chỉ thông qua các đặc trưng loại sự kiện và trạng thái thực thể, sử dụng danh tính tiến trình như một sự điều biến bổ sung thay vì một lỗi tắt.

Biểu diễn sự kiện cuối cùng. Ba thành phần được kết hợp và chuẩn hóa:

$$\mathbf{f}_e = \text{LayerNorm}([\mathbf{e}_{\text{mod}} \parallel \mathbf{c}]) \in \mathbb{R}^{2d} \quad (7)$$

trong đó $\parallel$ biểu thị phép nối. Loại sự kiện (được điều biến bởi danh tính tiến trình) chiếm d chiều đầu tiên; các đặc trưng liên tục chiếm d chiều thứ hai.

4.3 Ngân hàng Trạng thái Thực thể

Ngân hàng `EntityStateBank` lưu trữ hai tensor — một ma trận trạng thái $\mathbf{S} \in \mathbb{R}^{|\mathcal{V}| \times d}$, trong đó hàng $\mathbf{S}[v]$ giữ

trạng thái hiện tại $\mathbf{s}_v$ của thực thể v , và một vectơ dấu thời gian nhìn thấy lần cuối $\boldsymbol{\tau} \in \mathbb{R}^{|\mathcal{V}|}$, trong đó $\boldsymbol{\tau}[v]$ ghi lại thời gian gần đây nhất mà thực thể v tham gia vào bất kỳ sự kiện nào — trong CPU system RAM với bộ nhớ khóa trang (page-locked/pinned memory). Các tensor này được cố ý thiết kế để không được đăng ký làm bộ đệm PyTorch (PyTorch buffers), do đó lệnh `model.to(device)` không chuyển chúng sang GPU. Thiết kế này cho phép mở rộng quy mô lên các quần thể thực thể có thể làm cạn kiệt GPU VRAM (ví dụ: 176 triệu thực thể thô trong tập dữ liệu TRACE trước khi lập chỉ mục lại). Ở mỗi chunk, chỉ khoảng $3C$ trạng thái thực thể thực sự được chạm bởi các sự kiện của chunk (trong đó C là kích thước chunk) được thu thập sang GPU thông qua các truyền tải bộ nhớ ghim không chặn (non-blocking pinned-memory transfers), được xử lý qua SSM, và phân tán ngược lại. Cả hai tensor đều được khởi tạo về 0 (trạng thái) và -1 (dấu thời gian, biểu thị chưa thấy) vào đầu mỗi epoch đào tạo và duy trì qua các chunk sự kiện trong epoch. Việc thiết lập lại tại ranh giới epoch giúp mô hình không bị quá khớp vào quỹ đạo trạng thái cụ thể tích lũy ở các epoch trước, trong khi sự duy trì nội-epoch cung cấp bối cảnh tầm xa cần thiết cho luận điểm.

Giới hạn bộ nhớ. Ngân hàng yêu cầu $|\mathcal{V}| \times (d+1)$ giá trị dấu phẩy động trong bộ nhớ RAM hệ thống. Đối với tập dữ liệu Theia ($|\mathcal{V}| \approx 7.1 \times 10^6$, $d = 256$), bộ nhớ này chiếm khoảng 7.3 GB — hoàn toàn nằm trong mức 64 GB RAM hệ thống của một máy chủ thông thường, và độc lập với các giới hạn GPU VRAM. Việc truyền dữ liệu sang GPU ở mỗi chunk liên quan đến tối đa $3C$ trạng thái thực thể (ví dụ: $3 \times 16,384 = 49,152$ vectơ, ~ 48 MB với $d = 256$), là không đáng kể so với bộ nhớ GPU.

Truy xuất trạng thái và thời gian trôi qua. Đối với một sự kiện đến e_t với các khe thực thể $(v_{\text{subj}}, v_{\text{obj}}, v_{\text{obj2}})$, mô hình thu thập một tensor trạng thái $\mathbf{S}_e \in \mathbb{R}^{3 \times d}$ bằng cách lập chỉ mục vào $\mathbf{S}$. Một mặt nạ hợp lệ $\mathbf{m} \in \{0, 1\}^3$ được đặt thành 0 cho các thực thể vắng mặt: các siêu cạnh kích thước 2 lưu $v_{\text{obj2}} = -1$, được kẹp lại tại chỉ mục 0 trong quá trình truy xuất. Mặt nạ đảm bảo rằng trạng thái đọc sai tại chỉ mục 0 sẽ đóng góp 0 vào cả tính toán tập hợp và đường ghi lại, ngăn sự phá hỏng trạng thái của thực thể 0.

Thời gian trôi qua cho mỗi thực thể được tính như sau:

$$\Delta t_v = \max(0, t_{\text{curr}} - \boldsymbol{\tau}[v]) \quad (8)$$

trong đó $\Delta t_v = 0$ cho các thực thể chưa thấy trước đó ($\boldsymbol{\tau}[v] = -1$). Giá trị tối thiểu ở mức 0 ngăn ngừa lỗi dấu thời gian không đơn điệu do sắp xếp lại bên trong phân đoạn (shard). Tất cả các dấu thời gian được chuyển đổi sang giây so với một tham chiếu toàn cục t_0 (dấu thời gian tính bằng nano giây của sự kiện đầu tiên trong tập

đào tạo):

$$t_{\text{curr}} = (\text{timestamp_nanos}(e_t) - t_0) / 10^9 \quad (9)$$

Tất cả các tập phân chia (đào tạo, xác thực, kiểm tra) chia sẻ cùng t_0 để Δt_v vẫn hợp lệ qua ranh giới đào tạo-kiểm tra trong quá trình đánh giá khởi động ấm (warm-start) (Mục 4.7).

Thời gian trôi qua được chuyển đến các giai đoạn hạ nguồn ở dạng tỷ lệ logarit là $\log(1 + \Delta t_v)$ để nén dải động: khoảng cách giữa các sự kiện trong tập dữ liệu Theia kéo dài từ micro giây đến hàng giờ.

Chuẩn hóa thực thể trung tâm. Các thực thể có bậc cao (ví dụ: `init` với PID 1, tham gia vào hơn 1.5×10^5 sự kiện trong tập dữ liệu Theia) tích lũy các vectơ trạng thái với chuẩn không giới hạn thông qua các bản cập nhật SSM lặp lại. Không có chuẩn hóa, các phép chiếu khóa trong giai đoạn tập hợp (Mục 4.4) tạo ra điểm số chú ý gây tràn `float32 softmax` ($\exp(x)$ cho $x > 88$). Để ngăn chặn điều này, các trạng thái truy xuất được chuẩn hóa thông qua chuẩn hóa lớp (layer normalization) trước khi vào giai đoạn tập hợp. Việc chuẩn hóa này chỉ được áp dụng cho bản sao đã truy xuất $\hat{\mathbf{S}}_e$; ngân hàng $\mathbf{S}$ giữ lại các trạng thái chưa chuẩn hóa:

$$\hat{\mathbf{S}}_e = \text{LayerNorm}(\mathbf{S}_e) \odot \mathbf{m} \quad (10)$$

Mặt nạ hợp lệ $\mathbf{m}$ được áp dụng sau khi chuẩn hóa để đảm bảo các khe thực thể vắng mặt đóng góp 0 vào các tính toán hạ nguồn. Tất cả các tham chiếu tiếp theo đến các trạng thái thực thể trong một lượt truyền thẳng đơn lẻ đều đề cập đến $\hat{\mathbf{S}}_e$.

Ghi lại trạng thái. Sau bước cập nhật trạng thái SSM (Mục 4.5), các trạng thái được cập nhật được phân tán ngược lại vào $\mathbf{S}$ thông qua phép gán lập chỉ mục tại chỗ (in-place indexed assignment), và τ được cập nhật với t_{curr} . Chỉ các thực thể hợp lệ (những thực thể có $\mathbf{m}_v = 1$) mới được ghi lại. Để ngăn ngừa lỗi số học lan truyền qua các chunk, các giá trị NaN trong trạng thái được cập nhật được thay thế bằng 0 trước khi ghi lại.

4.4 Tập hợp Siêu cạnh ($\mathcal{V} \rightarrow \mathcal{E}$)

Cho trạng thái thực thể đã chuẩn hóa $\hat{\mathbf{S}}_e$ (Phương trình 10) và biểu diễn sự kiện $\mathbf{f}_e$ (Phương trình 7), giai đoạn tập hợp tạo ra một biểu diễn siêu cạnh duy nhất $\mathbf{x}_e \in \mathbb{R}^d$ kết hợp các trạng thái của 2-3 thực thể tham gia, được điều kiện hóa bởi ngữ nghĩa sự kiện. Việc tập hợp xác định mức độ mà lịch sử tích lũy của mỗi thực thể đóng góp vào biểu diễn của sự kiện hiện tại, trực tiếp kiểm soát thông tin có sẵn cho bộ phân loại.

Nhúng vai trò. Mỗi khe thực thể được gán một nhúng vai trò học được $\mathbf{r}_v \in \mathbb{R}^d$ từ bảng tra cứu gồm 3 mục (Chủ thể, Đối tượng Chính, Đối tượng Phụ). Các nhúng vai trò là cần thiết bởi vì việc tập hợp hoạt động trên

một tập hợp trạng thái không có thứ tự; nếu không có thông tin vai trò vị trí, cơ chế chú ý không thể phân biệt chủ thể với đối tượng, từ đó loại bỏ cấu trúc có hướng của sự kiện.

Bộ mô tả thực thể. Đối với mỗi thực thể $v \in V_{e_t}$, chúng tôi xây dựng một bộ mô tả $\mathbf{h}_v$ bằng cách nối trạng thái chuẩn hóa, nhúng vai trò và thời gian trôi qua tỷ lệ logarit:

$$\mathbf{h}_v = [\hat{\mathbf{s}}_v \parallel \mathbf{r}_v \parallel \log(1 + \Delta t_v)] \in \mathbb{R}^{2d+1} \quad (11)$$

Chú ý động. Các đặc trưng sự kiện $\mathbf{f}_e$ đóng vai trò là truy vấn, và các bộ mô tả thực thể đóng vai trò là khóa và giá trị. Điều này điều kiện hóa sự tập hợp vào loại sự kiện: đối với một `EVENT_WRITE`, đối tượng (tập đang được ghi) có thể mang nhiều trạng thái phân biệt hơn chủ thể, trong khi đối với một `EVENT_MMAP`, đối tượng phụ (vùng bộ nhớ) mang tín hiệu được xác định trong Mục 2.2. Quá trình tính toán chú ý là:

$$\mathbf{q} = \mathbf{W}_q \mathbf{f}_e \in \mathbb{R}^d \quad (12)$$

$$\mathbf{k}_v = \mathbf{W}_k \mathbf{h}_v \in \mathbb{R}^d \quad (13)$$

$$\mathbf{v}_v = \mathbf{W}_v \mathbf{h}_v \in \mathbb{R}^d \quad (14)$$

trong đó $\mathbf{W}_q \in \mathbb{R}^{d \times 2d}$, $\mathbf{W}_k, \mathbf{W}_v \in \mathbb{R}^{d \times (2d+1)}$. Trọng số chú ý được tính qua tích vô hướng có tỷ lệ và mặt nạ. Đối với mỗi khe thực thể v , điểm số thô là:

$$s_v = \mathbf{q}^\top \mathbf{k}_v / \sqrt{d} \quad (15)$$

Các khe không hợp lệ ($\mathbf{m}_v = 0$) có điểm được thiết lập thành $-\infty$ trước softmax, do đó $\exp(-\infty) = 0$ ở cả tử số và mẫu số:

$$\alpha_v = \begin{cases} \frac{\exp(s_v)}{\sum_{u: \mathbf{m}_u=1} \exp(s_u)} & \text{nếu } \mathbf{m}_v = 1 \\ 0 & \text{nếu } \mathbf{m}_v = 0 \end{cases} \quad (16)$$

Như một biện pháp bảo vệ số học, bất kỳ giá trị NaN nào trong các trọng số chú ý (có thể phát sinh từ các sự kiện suy biến khi tất cả các khe bị che lấp, tạo ra 0/0 trong softmax) đều được thay thế bằng 0. Biểu diễn siêu cạnh lúc đó là:

$$\mathbf{x}_e = \mathbf{W}_{\text{out}} \sum_{v \in V_{e_t}} \alpha_v \mathbf{v}_v \in \mathbb{R}^d \quad (17)$$

trong đó $\mathbf{W}_{\text{out}} \in \mathbb{R}^{d \times d}$ là một phép chiếu đầu ra tuyến tính. Nhúng vai trò $\mathbf{r}_v$ được giữ lại và chuyển qua giai đoạn cập nhật SSM (Mục 4.5), tại đây chúng đảm bảo sự truyền bá trạng thái đặc thù cho từng vai trò.

4.5 Cập nhật Trạng thái SSM Chọn lọc ($\mathcal{E} \rightarrow \mathcal{V}$)

Được cung cấp biểu diễn siêu cạnh $\mathbf{x}_e$ (Phương trình 17) và nhúng vai trò $\mathbf{r}_v$ từ giai đoạn tập hợp, bộ cập nhật

SSM tính toán trạng thái mới $\mathbf{s}_v^+$ cho từng thực thể tham gia $v \in V_{e_t}$. Đây là cơ chế mà thông qua đó bối cảnh hành vi tích lũy truyền qua các thực thể và theo thời gian: trạng thái được cập nhật mã hóa cả lịch sử trước đó của thực thể (thông qua phần suy giảm) và đóng góp của sự kiện hiện tại (thông qua phần đầu vào).

Ma trận suy giảm trạng thái. Ma trận suy giảm trạng thái đường chéo $A \in \mathbb{R}^d$ được lưu trữ trong không gian log như một tham số có thể học được $\mathbf{A}_{\log} \in \mathbb{R}^d$ và được phục hồi như sau:

$$A = -\exp(\text{clamp}(\mathbf{A}_{\log}, -4, 4)) \quad (18)$$

Tất cả các thành phần của A đều âm nghiêm ngặt, đảm bảo các trạng thái phân rã theo thời gian khi không có đầu vào mới. $\mathbf{A}_{\log}$ được khởi tạo bằng $\log(\text{linspace}(0.1, 2.0, d))$, tạo ra tỷ lệ suy giảm ban đầu kéo dài $A_j \in [-2.0, -0.1]$. Sự khởi tạo đa quy mô thời gian (multi-timescale) này gán cho các chiều trạng thái khác nhau các tỷ lệ suy giảm khác nhau: các chiều phân rã chậm ($A_j \approx -0.1$) giữ thông tin qua nhiều sự kiện cho các phụ thuộc tầm xa, trong khi các chiều phân rã nhanh ($A_j \approx -2.0$) đáp ứng nhanh với hoạt động gần đây. Việc giới hạn $\mathbf{A}_{\log}$ ở $[-4, 4]$ ngăn tràn số học trong phép lấy mũ.

Phép chiếu đầu vào đặc thù theo vai trò. Ma trận đầu vào B được tính từ sự nối của biểu diễn siêu cạnh $\mathbf{x}_e$ và nhúng vai trò của thực thể $\mathbf{r}_v$:

$$B_v = \mathbf{W}_B[\mathbf{x}_e \parallel \mathbf{r}_v] \in \mathbb{R}^d \quad (19)$$

trong đó $\mathbf{W}_B \in \mathbb{R}^{d \times 2d}$. Việc bao gồm những vai trò tạo ra các giá trị B_v khác nhau cho chủ thể, đối tượng chính, và đối tượng phụ của cùng một sự kiện. Nếu không có điều này, cả ba thực thể sẽ nhận các hướng cập nhật giống hệt nhau từ $\mathbf{x}_e$, khiến các trạng thái của chúng hội tụ và phá hủy cấu trúc đồ thị có hướng phân biệt chủ thể với đối tượng.

Rời rạc hóa nhận biết thời gian. Các tham số SSM thời gian liên tục (A, B_v) được rời rạc hóa sử dụng kích thước bước phụ thuộc vào đầu vào $\Delta_v \in \mathbb{R}^d$:

$$\Delta_v = \text{clamp}(\text{softplus}(\mathbf{W}_\Delta[\mathbf{x}_e \parallel \mathbf{r}_v]) \cdot (1 + \lambda \log(1 + \Delta t_v))), \quad (20)$$

trong đó $\mathbf{W}_\Delta \in \mathbb{R}^{d \times 2d}$ với độ lệch (bias) khởi tạo ở -2.0 (tạo ra kích thước bước ban đầu là $\text{softplus}(-2) \approx 0.13$), $\lambda = 0.1$, và Δt_v là thời gian đã trôi qua từ Mục 4.3. Phép nhân chia tỷ lệ thời gian $(1 + 0.1 \cdot \log(1 + \Delta t_v))$ áp dụng một sự điều chỉnh nhẹ (khoảng 1.0 đến $1.7\times$ trên phạm vi quan sát được) giúp mở rộng bước rời rạc hóa cho các thực thể đã không hoạt động, cho phép sự suy giảm phản ánh thời gian thực đã trôi qua mà không gây ra sự mất ổn định gradient như việc nhân trực tiếp Δt_v có thể gây ra. Giới hạn tại 5 ngăn chặn sự tràn số học trong bước lũy thừa tiếp theo.

Các tham số rời rạc hóa là:

$$\bar{A}_v = \exp(\Delta_v \odot A) \in (0, 1)^d \quad (21)$$

$$\bar{B}_v = \Delta_v \odot B_v \quad (22)$$

trong đó $\bar{A}_v$ kiểm soát sự giữ lại trạng thái trước đó trên mỗi chiều (giá trị gần 1 giữ lại, gần 0 là quên đi), và $\bar{B}_v$ tỷ lệ hóa mức độ đóng góp của đầu vào. Phép cập nhật SSM thô là:

$$\tilde{\mathbf{s}}_v = \bar{A}_v \odot \hat{\mathbf{s}}_v + \bar{B}_v \odot \mathbf{x}_e \quad (23)$$

Kết nối thẳng dư có cổng. Một cổng học được $\mathbf{g}_v \in (0, 1)^d$ trộn lẫn kết quả của SSM với trạng thái trước đó:

$$\mathbf{g}_v = \sigma(\mathbf{W}_g[\mathbf{x}_e \parallel \mathbf{r}_v]) \quad (24)$$

$$\mathbf{s}_v^+ = \mathbf{g}_v \odot \tilde{\mathbf{s}}_v + (1 - \mathbf{g}_v) \odot \hat{\mathbf{s}}_v \quad (25)$$

trong đó $\mathbf{W}_g \in \mathbb{R}^{d \times 2d}$ với bias được khởi tạo là -3.0 , do đó $\mathbf{g}_v \approx 0.04$ lúc bắt đầu đào tạo. Việc khởi tạo gần đúng này là cần thiết về mặt cấu trúc: ở lần khởi tạo ngẫu nhiên, các tham số SSM (A, B) sinh ra các bản cập nhật trạng thái tùy ý, và một cổng mở sẽ ngay lập tức ghi đè lên các trạng thái thực thể bằng nhiễu (noise). Khởi tạo cổng gần bằng không buộc mô hình chủ động học khi nào nên truyền bá trạng thái, ngăn sự quá mượt (over-smoothing) ở bước khởi tạo. Cổng mở dần trong quá trình đào tạo khi các tham số SSM trở nên có ý nghĩa.

4.6 Đầu Phân loại

Giai đoạn cuối cùng quyết định xem sự kiện e_t có phải là một phần của chuỗi tấn công hay không bằng cách đánh giá cả ngữ nghĩa độc lập của nó và trạng thái hành vi tích lũy của các thực thể tham gia.

Định tuyến Gradient qua các trạng thái sau cập nhật. Bộ phân loại hoạt động trên trạng thái sau cập nhật $\mathbf{s}_v^+$ tính ở Mục 4.5, không phải trạng thái trước cập nhật được lấy từ ngân hàng. Định tuyến này cực kỳ quan trọng đối với cấu trúc cho việc đào tạo toàn trình (end-to-end). Các trạng thái của ngân hàng được coi là các hằng số tách rời trong quá trình truyền thẳng; nếu bộ phân loại đọc trực tiếp từ ngân hàng, biểu đồ tính toán sẽ kết thúc, cắt đứt hoàn toàn dòng gradient tới bộ cập nhật SSM (thứ tạo ra trạng thái mới) và tới bộ tập hợp siêu cạnh (thứ tạo ra đầu vào SSM). Bằng cách phân loại trên trạng thái sau cập nhật, gradient được truyền ngược qua toàn bộ chuỗi truyền thẳng.

Lấy trung bình có mặt nạ hợp lệ. Trạng thái sau cập nhật được gộp (pooled) vào một vectơ bối cảnh mức độ sự kiện duy nhất $\bar{\mathbf{s}} \in \mathbb{R}^d$. Quá trình này yêu cầu che

mặt nạ các đối tượng phụ vắng mặt (nơi $\mathbf{m}_v = 0$):

$$\bar{\mathbf{s}} = \text{LayerNorm} \left(\frac{1}{\sum_v \mathbf{m}_v} \sum_{v \in V_{e_t}} \mathbf{m}_v \cdot \mathbf{s}_v^+ \right) \quad (26)$$

Việc che mặt nạ trước khi cộng là bắt buộc do cập nhật SSM (Phương trình 23) sinh ra bản cập nhật khác 0 ngay cả đối với khe thực thể rỗng (được điều khiển bởi biểu diễn sự kiện $\mathbf{x}_e$ thông qua phép chiếu vai trò $\bar{B}_v$). Không có mặt nạ, khe rỗng sẽ làm hỏng giá trị trung bình. Trạng thái gộp được chuẩn hóa lớp để làm ổn định đầu vào bộ phân loại.

Xác suất phát hiện. Trạng thái đã gộp được nối với các đặc trưng sự kiện đã chiếu $\mathbf{W}_p \mathbf{f}_e$ (với $\mathbf{W}_p \in \mathbb{R}^{d \times 2d}$). Xác suất $\hat{y}_t \in [0, 1]$ cho sự kiện e_t là sự kiện tấn công được tính thông qua một Bộ Perceptron đa lớp (MLP):

$$\hat{y}_t = \sigma(\text{MLP}([\bar{\mathbf{s}} \parallel \mathbf{W}_p \mathbf{f}_e])) \quad (27)$$

MLP bao gồm ba lớp tuyến tính với hàm kích hoạt GELU trung gian và Dropout ($\rho = 0.1$) sau lớp đầu tiên, chiều số chiều từ $2d \rightarrow d \rightarrow d/2 \rightarrow 1$. Hàm mất mát Binary Cross-Entropy (BCE) được tính toán dựa trên nhãn thực tế của sự kiện $y_t \in \{0, 1\}$.

4.7 Đào tạo: TBPTT và Khởi động âm Liên tục

HyperMamba yêu cầu các kỹ thuật đào tạo khác với các mô hình NIDS không trạng thái vì các trạng thái thực thể tồn tại dai dẳng qua chuỗi sự kiện.

Lan truyền ngược qua Thời gian Bị cắt ngắn (TBPTT). Để xử lý các luồng sự kiện liên tục mà không làm cạn kiệt bộ nhớ, các sự kiện được phân tách theo thời gian thành các đoạn (chunk) kích thước $C = 4096$. Với mỗi chunk, mô hình thực hiện truyền thẳng (Mục 4.2-4.6) và tính tổn thất BCE. Quan trọng là, các trạng thái cập nhật được ghi vào **EntityStateBank** được giữ lại cho chunk tiếp theo, nhưng biểu đồ tính toán của chúng bị tách ra qua lệnh ‘detach()’. Chiến lược TBPTT này cho phép bối cảnh truyền thẳng tồn tại vô hạn trên toàn bộ tập dữ liệu, tích lũy các tín hiệu nhiễu bản tầm xa, trong khi giới hạn tính toán truyền ngược gradient ở mức $\mathcal{O}(C)$.

Tối ưu hóa và sự ổn định. Mô hình được tối ưu bằng Adam với lịch trình tỷ lệ học Cosine Annealing. Để xử lý sự mất cân bằng lớp nghiêm trọng trong các luồng kiểm toán, tổn thất BCE được tính theo tỷ trọng số lượng mẫu âm so với mẫu dương trong tập đào tạo (giới hạn ở mức 30.0 để tránh bùng nổ gradient). Vì các SSM hoạt động trên các luồng rời rạc đôi khi có thể tạo ra tính bất ổn số học, các chunk dẫn đến tổn thất NaN hoặc gradient NaN sẽ bị bỏ qua và ngân hàng trạng thái ngay lập tức bị tách ra (detach) để ngăn chặn làm hỏng

các chunk tiếp theo. Chuẩn gradient cũng được cắt gọn thêm ở mức 1.0.

Đặt lại ranh giới Epoch. **EntityStateBank** hoàn toàn được đưa về 0 ở đầu mỗi epoch đào tạo. Nếu không thiết lập lại, mô hình sẽ bắt đầu epoch k (đào tạo trên phân đoạn thời gian đầu tiên) sử dụng các trạng thái thực thể tương lai tích lũy ở cuối epoch $k - 1$ (sau khi đánh giá trên các shard xác thực). Điều này sẽ tạo ra một sự rò rỉ dữ liệu thời gian nghiêm trọng. Đặt lại đảm bảo mỗi epoch học cách tích lũy trạng thái hoàn toàn từ quá khứ.

Đánh giá khởi động âm liên tục. Trong một triển khai thực tế, một NIDS hoạt động liên tục; các trạng thái thực thể không bao giờ “lạnh” (trống). Để đánh giá chính xác khả năng phát hiện trên một phân đoạn kiểm tra, chúng ta phải mô phỏng triển khai liên tục này. Trước khi đánh giá tập kiểm tra, mô hình chạy quá trình khởi động (warmup) chỉ truyền thẳng (tắt gradient và việc thu thập số liệu) đi qua tất cả các phân đoạn đào tạo và xác thực trước đó theo thứ tự thời gian nghiêm ngặt để lấp đầy **EntityStateBank**. Sau đó, tập kiểm tra được đánh giá sử dụng các trạng thái đã “ấm” này. Việc đánh giá một mô hình có trạng thái từ khởi động lạnh (cold start) sẽ đánh giá thấp một cách hệ thống khả năng phát hiện các cuộc tấn công nhiều giai đoạn bắt đầu từ trước cửa sổ kiểm tra.

Lựa chọn ngưỡng. Phù hợp với các tiêu chuẩn đánh giá nghiêm ngặt cho học máy trong bảo mật máy tính [1], ngưỡng quyết định được lựa chọn động để tối đa hóa F1-score độc quyền trên tập xác thực. Ngưỡng cố định này sau đó được áp dụng cho tập kiểm tra, ngăn ngừa hoàn toàn sự rò rỉ ngưỡng của tập kiểm tra.

5 Đánh giá

5.1 Thiết lập Thử nghiệm

Tập dữ liệu. Chúng tôi đánh giá HyperMamba trên hai tập dữ liệu từ bộ chuẩn DARPA Transparent Computing (TC) Engagement 3, một môi trường thử nghiệm tiêu chuẩn cho NIDS dựa trên nguồn gốc:

- E3-Theia (Linux/Firefox backdoor): 25 phân đoạn theo trình tự thời gian (shards), khoảng 105 triệu sự kiện, trải dài trên hai chiến dịch APT đa giai đoạn riêng biệt. Được đánh giá theo nhãn xuyên tiến trình (crossprocess - chỉ các sự kiện của tiến trình con, tỷ lệ mẫu dương <1%).
- E3-CADETS (FreeBSD/Nginx backdoor): 10 phân đoạn theo trình tự thời gian, khoảng 27 triệu sự kiện, với một vectơ tấn công khác biệt (xâm nhập máy chủ web Nginx). Được đánh giá theo nhãn rộng (tất cả các nút liên quan đến cuộc tấn công)

để so sánh trực tiếp với các phương pháp cơ sở đã công bố.

Đường ống tiền xử lý. Nhật ký JSON CDM thô được nạp vào các phân đoạn Parquet theo trình tự thời gian, mỗi phân đoạn chứa các sự kiện kiểm toán có cấu trúc với UUID của chủ thể, đối tượng và đối tượng phụ tùy chọn. Mỗi sự kiện được đại diện bởi một vectơ đặc trưng 35 chiều: 18 chỉ số mã hóa one-hot của loại sự kiện (ví dụ: `EVENT_READ`, `EVENT_WRITE`, `EVENT_EXECUTE`, `EVENT_FORK`) và 17 đặc trưng bối cảnh. Các đặc trưng bối cảnh bao gồm 8 giá trị liên tục — giờ trong ngày, số thực thể kết nối của siêu cạnh (2 hoặc 3), độ hiếm của loại sự kiện, kích thước byte của sự kiện, khoảng cách thời gian giữa các sự kiện cho cùng một chủ thể, độ sâu đường dẫn đối tượng, tần suất sự kiện đối tượng được chuyển đổi logarit, và độ mới của chủ thể/đối tượng — và 9 chỉ số nhị phân cho sự hiện diện của đường dẫn, loại đối tượng (tệp, netflow, bộ nhớ), cờ đường dẫn nhảy cảm (`/tmp`, `/home`, `/var/log`), và tính mới của cặp chủ thể-đối tượng. Các đặc trưng liên tục được chuẩn hóa Z-score sử dụng số liệu thống kê được tính toán độc quyền trên các phân đoạn đào tạo thông qua thuật toán trực tuyến của Welford để ngăn chặn rò rỉ dữ liệu thời gian; các đặc trưng nhị phân được tự động phát hiện và bỏ qua trong quá trình chuẩn hóa.

Từ vựng thực thể và tiến trình. Tất cả các UUID thực thể duy nhất trên toàn bộ các phân đoạn được thu thập (ngoại trừ lính canh null CDM), được sắp xếp theo thứ tự từ điển và được gán các ID số nguyên liên tiếp $0, \dots, |\mathcal{V}| - 1$. Mỗi thực thể được gán thể là chỉ chủ thể, chỉ đối tượng, hoặc cả hai dựa trên vai trò quan sát được của nó. Từ vựng Theia chứa khoảng 7.1 triệu thực thể; CADETS chứa khoảng 1.5 triệu thực thể. Đối với từ vựng tên tiến trình, đường dẫn tiến trình hiệu dụng của mỗi chủ thể được trích xuất từ trường `process_path` (hoặc nếu thiếu, là mã thông báo đầu tiên của `cmd_line`). 50 đường dẫn thường gặp nhất tạo thành từ vựng (chỉ mục 1–50), với chỉ mục 0 dành cho đường dẫn không xác định/thiếu và chỉ mục 51 dành cho các đường dẫn còn lại ở đuôi dài, tạo ra tổng cộng 52 lớp.

Tập phân chia. Việc đánh giá được thực hiện nghiêm ngặt theo trình tự thời gian để tránh rò rỉ dữ liệu thời gian. Đối với Theia, dữ liệu bao gồm hai chiến dịch APT đa giai đoạn riêng biệt. Chiến dịch 1 trải dài trên các phân đoạn 0–10, và Chiến dịch 2 trải dài trên các phân đoạn 12–24. Để đánh giá cả sự tiếp tục trong cùng chiến dịch và sự tổng quát hóa chéo chiến dịch, chúng tôi xác định hai chế độ đánh giá:

- Chiến dịch hỗn hợp (**full**): Cả hai chiến dịch đều có mặt trong tập đào tạo. Chúng tôi gán các phân đoạn $\{0..7, 12..19\}$ cho đào tạo, các phân đoạn $\{8, 9, 20, 21\}$ cho xác thực, và các phân đoạn

$\{10, 22..24\}$ cho kiểm tra. Điều này khớp với quy trình đánh giá tiêu chuẩn của các phương pháp cơ sở của chúng tôi.

- Chéo chiến dịch (**cross**): Kiểm tra khả năng tổng quát hóa đối với các chiến lược tấn công hoàn toàn chưa từng thấy. Mô hình được đào tạo độc quyền trên Chiến dịch 1 (các phân đoạn đào tạo $\{0..8\}$, phân đoạn xác thực $\{9, 10\}$) và được kiểm tra độc quyền trên Chiến dịch 2 (các phân đoạn kiểm tra $\{12..24\}$).

Đối với CADETS, chúng tôi sử dụng phân chia chéo chiến dịch: các phân đoạn 0–5 cho đào tạo, các phân đoạn 6–7 cho xác thực, và các phân đoạn 8–9 cho kiểm tra, dưới nhãn tấn công rộng. Hình 3 trực quan hóa cấu trúc phân đoạn và ranh giới phân chia cho tất cả các tập dữ liệu.

Phương pháp cơ sở. Chúng tôi so sánh HyperMamba với các NIDS dựa trên nguồn gốc tiên tiến nhất, định vị từng hệ thống theo độ phân giải phát hiện cơ bản của chúng:

- KAIROS [3]: Hoạt động ở cấp độ đồ thị, gán điểm dị thường cho toàn bộ các đồ thị nguồn gốc vĩ mô.
- MAGIC [7]: Hoạt động ở cấp độ đồ thị, phát hiện các cuộc tấn công thông qua lỗi tái cấu trúc của các đồ thị con cấu trúc.
- FLASH [8]: Hoạt động ở cấp độ nút, gán cờ các tiến trình hoặc tệp dị thường.
- ThreaTrace [9]: Hoạt động ở cấp độ nút, phân loại các thực thể bên trong đồ thị nguồn gốc.

Quan trọng là, không có phương pháp cơ sở nào trong số này có khả năng phát hiện ở cấp độ sự kiện một cách nguyên bản, vốn là độ phân giải nhỏ nhất và là đầu ra chính của HyperMamba.

Số liệu và phương pháp xác định ngưỡng. Chúng tôi báo cáo Area Under the Precision-Recall Curve (AUPRC) như số liệu chính không phụ thuộc ngưỡng, do sự mất cân bằng lớp nghiêm trọng của các luồng kiểm toán. Để tính toán các số liệu phụ thuộc vào ngưỡng (F1-score và Tỷ lệ Dương tính Giả - FPR), chúng tôi tuân thủ các hướng dẫn nghiêm ngặt cho học máy bảo mật được đề xuất bởi Arp và cộng sự [1]: các ngưỡng được chọn động từ dự đoán trên tập xác thực bằng điểm vận hành tối đa hóa F1, và các giá trị cố định này sau đó được áp dụng cho dữ liệu kiểm tra được giữ lại. Điều này ngăn chặn nghiêm ngặt sự rò rỉ tối ưu hóa ngưỡng của tập kiểm tra. Các số liệu được báo cáo ở cả cấp độ sự kiện (đối với HyperMamba) và cấp độ nút (đối với HyperMamba và các phương pháp cơ sở).

Chi tiết triển khai. HyperMamba được triển khai trong PyTorch. Chúng tôi sử dụng trình tối ưu hóa

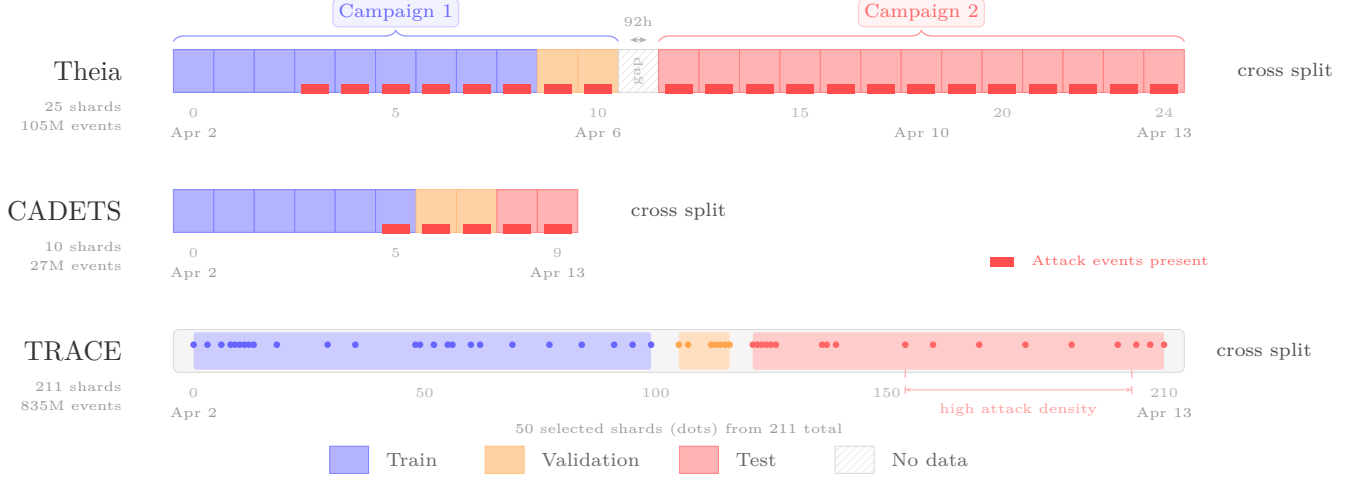


Figure 3: Chronological shard structure and cross-campaign evaluation splits. Each rectangle represents one data shard. Theia (top): the model trains on Campaign 1 (shards 0–8) and is tested entirely on the unseen Campaign 2 (shards 12–24), separated by a 92-hour gap. CADETS (middle): 10 shards with attacks concentrated in later shards. TRACE (bottom): 50 strategically selected shards from 211 total ($\sim 200\text{M}$ of 835M events); dots mark selected shards. Small red bars indicate shards containing attack events. All splits are strictly chronological to prevent temporal data leakage.

Adam ($\beta_1=0.9$, $\beta_2=0.999$) với tốc độ học 5×10^{-4} , suy giảm trọng số (weight decay) 10^{-4} , và cosine annealing về $\eta_{\min} = 10^{-5}$. Chiều mô hình là $d = 256$ với chú ý tích vô hướng một đầu (single-head dot-product attention) trong bộ tập hợp siêu cạnh. Dropout của bộ phân loại là 0.1; dropout danh tính tiến trình là 0.3 (Mục 4.2). Quá trình đào tạo chạy trong 10 epoch với dừng sớm dựa trên AUPRC xác thực (kiên nhẫn 5). Kích thước chunk cho TBPTT là $C = 16,384$ sự kiện. Trọng số lớp dương trong hàm mất mát BCE được giới hạn ở mức 30. Tất cả các thử nghiệm đều sử dụng seed 42 với các cài đặt PyTorch tắt định. Quá trình đào tạo và suy luận được thực hiện trên một GPU NVIDIA A100 40 GB duy nhất với 64 GB RAM hệ thống.

5.2 Kết quả Chính (C4)

So sánh số lượng trực tiếp giữa các chỉ số cấp sự kiện của HyperMamba và các NIDS dựa trên nguồn gốc hiện tại là không hợp lệ về mặt cấu trúc: KAIROS và MAGIC tạo ra một điểm dị thường cho mỗi cửa sổ phân tích, FLASH và ThreaTrace tạo ra một điểm cho mỗi nút, và không hệ thống nào tạo ra các phân loại trên từng sự kiện. Việc lan truyền điểm dị thường của một nút tới tất cả các sự kiện cấu thành của nó — sự thích ứng khả thi duy nhất — dẫn đến sự sụp đổ nghiêm trọng về độ chính xác: một nút bị gắn cờ đã tạo ra 50.000 sự kiện trong suốt thời gian tồn tại của nó chỉ có tối đa một số ít sự kiện tấn công xuyên tiến trình thực sự, vì vậy bất kỳ ngưỡng nào đạt được độ bao phủ (recall) đều sẽ gắn

cờ hàng chục ngàn sự kiện lành tính là độc hại. Đây không phải là thất bại của các hệ thống cụ thể mà là hệ quả cấu trúc của việc hoạt động ở mức độ chi tiết thô hơn. Do đó, chúng tôi chỉ báo cáo hiệu suất của các phương pháp cơ sở ở các mức chi tiết nguyên bản của chúng, và so sánh HyperMamba ở cả cấp độ sự kiện (đầu ra nguyên bản của nó) và cấp độ nút (để tương thích với các công trình trước đây).

Nhãn xuyên tiến trình (crossprocess) đánh giá việc phát hiện nhóm phụ tấn công khó khăn nhất về mặt vận hành: các tiến trình con ở giai đoạn sau có các lệnh gọi hệ thống riêng lẻ không thể phân biệt được về mặt thống kê với hoạt động lành tính. Tiến trình điểm vào có hành vi khai thác ban đầu dị thường được loại trừ rõ ràng. Cả báo cáo của bốn hệ thống cơ sở đều được đánh giá dựa trên nhãn tấn công rộng hơn bao gồm các sự kiện điểm vào này; do đó, đánh giá xuyên tiến trình của chúng tôi là khó khăn hơn nghiêm ngặt. Để cho phép so sánh trực tiếp với nghiên cứu trước đây, chúng tôi đánh giá thêm HyperMamba theo một giao thức thống nhất gần giống với các điều kiện đánh giá của KAIROS và ThreaTrace: dữ liệu đào tạo trước ngày 10 tháng 4, tất cả các nút tấn công được gắn nhãn là mẫu dương, phân tách kiểm tra theo trình tự thời gian.

Ở cấp độ nút — mức độ chi tiết nhỏ nhất mà các phương pháp cơ sở hoạt động — HyperMamba được đánh giá theo giao thức KAIROS đạt được AUPRC nút là 0.9987 và F1 là 0.9926, vượt qua cả bốn phương pháp cơ sở bao gồm KAIROS (0.818), ThreaTrace (0.930), FLASH (0.960), và tương đương với MAGIC (0.999).

Table 1: Hiệu suất phát hiện trên DARPA TC E3. Các phương pháp cơ sở sử dụng nhân tấn công rộng (tất cả các nút tấn công); các dòng xuyên tiến trình của HyperMamba sử dụng nhân xuyên tiến trình (crossprocess+) nghiêm ngặt hơn (loại trừ các tiến trình điểm vào). Các dòng giao thức KAIROS và CADETS sử dụng nhân rộng để so sánh trực tiếp. AUPRC nút 0.9987 của dòng giao thức KAIROS phản ánh tỷ lệ mẫu dương 32% của nhân rộng, bao gồm hành vi điểm vào dị thường vốn tương đối dễ phát hiện; các dòng xuyên tiến trình sử dụng tỷ lệ mẫu dương <1%.

Mô hình	Nhân	Độ phân giải	AUPRC Sự kiện	AUPRC Nút	F1 Nút	FPR Nút
KAIROS [3]	Broad	Graph	—	[†] 0.818	[†] —	[†] —
MAGIC [7]	Broad	Graph	—	[†] 0.999	[†] —	[†] —
FLASH [8]	Broad	Node	—	[†] 0.960	[†] 0.970	[†] —
ThreaTrace [9]	Broad	Node	—	[†] 0.930	[†] 0.930	[†] —
HyperMamba (Giao thức KAIROS)	Broad	Event	0.9972	0.9987	0.9926	0.0066
HyperMamba (Cùng chiến dịch)	Crossprocess	Event	0.8523	0.9172	0.8460	< 0.0001
HyperMamba (Chéo chiến dịch)	Crossprocess	Event	0.7586	0.8939	0.9164	0.0022
E3-CADETS (FreeBSD / Nginx backdoor)						
HyperMamba (CADETS Chéo chiến dịch)	Broad	Event	0.9707	0.9973	0.9969	< 0.0001

[†] Kết quả đã công bố trên DARPA TC E3 Theia được đánh giá dựa trên nhân tấn công rộng bao gồm cả các tiến trình điểm vào.

Nhân tấn công rộng bao gồm các tiến trình điểm vào (27,379 trong số 84,615 nút chủ thể, tỷ lệ dương tính 32%), làm cho nó trở thành một bài toán phân loại tương đối dễ dàng mà HyperMamba giải quyết gần như hoàn hảo. Quan trọng là, HyperMamba đồng thời cung cấp AUPRC cấp sự kiện là 0.9972 trên chính đánh giá này — mức độ chính xác mà không hệ thống cơ sở nào có thể tạo ra ở bất kỳ độ phân giải nào.

Đánh giá có ý nghĩa vận hành thực tế là nhân xuyên tiến trình, nơi các sự kiện tấn công chiếm chưa đầy 1% tổng số sự kiện và các tiến trình điểm vào được loại trừ. Dưới nhân nghiêm ngặt hơn này, HyperMamba đạt được AUPRC cấp sự kiện là 0.8523 (cùng chiến dịch) và 0.7586 (chéo chiến dịch), với tỷ lệ dương tính giả lần lượt là < 0.0001 và 0.0011. Như một cận dưới cho FPR cấp sự kiện của phương pháp cơ sở: ThreaTrace gần cỡ khoảng 310 nút chủ thể độc hại trên tập kiểm tra Theia. Các nút này cùng nhau tạo ra khoảng 4.2 triệu sự kiện, trong đó có khoảng 36,000 sự kiện xuyên tiến trình thực sự. Việc lan truyền điểm số nút của ThreaTrace tới tất cả các sự kiện cấu thành ở bất kỳ ngưỡng nào đạt được recall > 0.9 sẽ gần cỡ cho toàn bộ 4.2 triệu sự kiện, tạo ra một FPR cấp sự kiện vượt quá 0.99. Độ chính xác cấp sự kiện có ý nghĩa là không thể đạt được về mặt cấu trúc thông qua việc lan truyền điểm số.

Tổng quát hóa chéo tập dữ liệu (E3-CADETS). Để xác nhận rằng HyperMamba có thể tổng quát hóa vượt ra ngoài một tập dữ liệu và vectơ tấn công duy nhất, chúng tôi đánh giá trên tập dữ liệu DARPA TC E3 CADETS (FreeBSD, cửa sau Nginx) dưới nhân tấn công rộng trong môi trường chéo chiến dịch. HyperMamba đạt được AUPRC cấp sự kiện là 0.9707 (AUROC 0.9997) và AUPRC cấp nút là 0.9973 với F1 nút là 0.9969 và FPR < 0.0001. Hiệu suất ở cấp độ nút tương đương hoặc vượt qua kết quả AUPRC nút 0.999 đã công bố của MAGIC trên CADETS, trong khi HyperMamba cung cấp thêm quy kết ở cấp độ sự kiện mà không hệ thống

nào trước đây cung cấp. Hình 4(a) trực quan hóa đặc tính vận hành Precision-Recall trên các tập dữ liệu và phân tích.

5.3 Nghiên cứu Cắt bỏ: Đóng góp của các Thành phần

Chúng tôi hệ thống hóa việc cắt bỏ từng thành phần kiến trúc để cô lập đóng góp của nó. Tất cả các nghiên cứu cắt bỏ đều sử dụng tập phân chia chéo chiến dịch (đào tạo Chiến dịch 1, kiểm tra Chiến dịch 2) với nhân xuyên tiến trình — thiết lập đánh giá khó nhất.

Thiết kế cắt bỏ. **no_cross_entity**: bộ tập hợp siêu cạnh bị tắt; trạng thái SSM của mỗi thực thể được cập nhật chỉ bằng cách sử dụng trạng thái trước đó của chính nó và các đặc trưng sự kiện, mà không quan sát các thực thể đồng tham gia.

no_state: ngăn hàng trạng thái thực thể bị ngắt kết nối về mặt cấu trúc; bộ phân loại nhận các vectơ không thay thế cho trạng thái tích lũy, chỉ giữ lại bộ mã hóa sự kiện (loại sự kiện, danh tính tiến trình, đặc trưng liên tục).

no_process_identity: toàn bộ đường dẫn danh tính tiến trình có cổng bị loại bỏ — cả việc tra cứu nhúng tiến trình và cổng học được g_p đều bị tắt, do đó mô hình nhận một vectơ không thay thế cho số hạng điều biến tiến trình trong Phương trình 5. Mô hình chỉ nhìn thấy các loại sự kiện, đặc trưng liên tục và trạng thái tích lũy.

no_state + no_process: cả trạng thái và danh tính tiến trình được loại bỏ đồng thời, cô lập sự đóng góp của riêng loại sự kiện và các đặc trưng liên tục.

Kết quả. Bảng 2 và Hình 5 tiết lộ một phân cấp rõ ràng về đóng góp của các thành phần.

Danh tính tiến trình (AUPRC sự kiện giảm -55.1%) là thành phần đóng góp đơn lẻ vượt trội cho phát hiện chéo chiến dịch. Nếu không có nhúng đường dẫn tiến

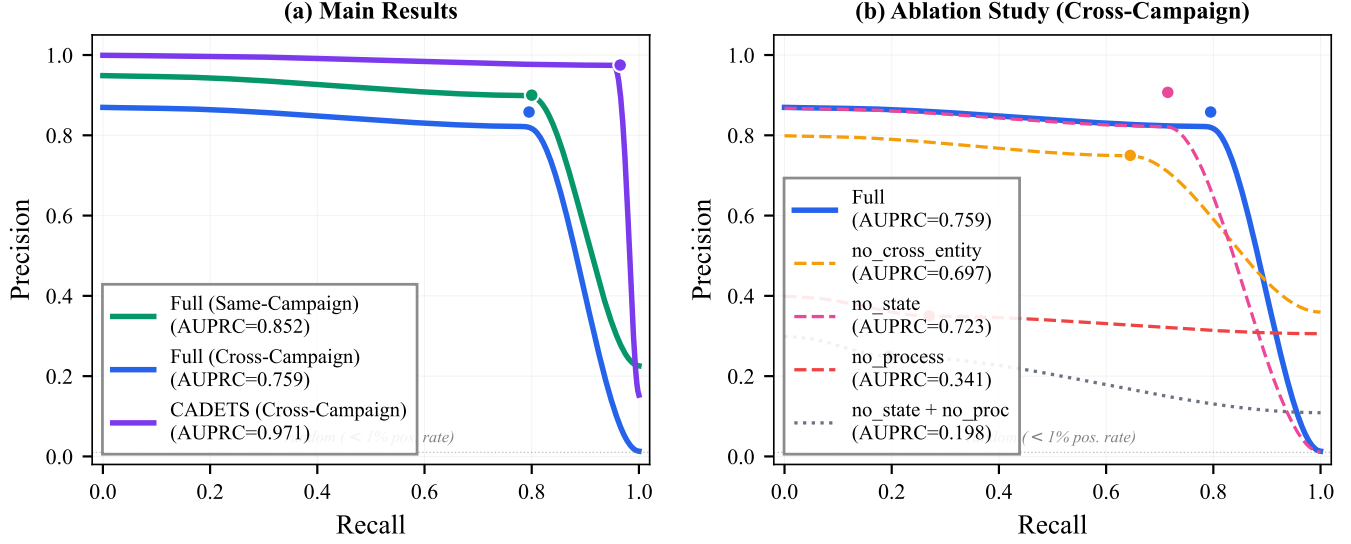


Figure 4: Precision-Recall curves. Dots mark the F1-optimal operating point (threshold selected from validation). (a) Main results: CADETS achieves near-perfect precision across all recall levels; the Theia cross-campaign curve shows the expected performance gap from same-campaign, reflecting the difficulty of generalizing to unseen attack strategies. (b) Ablation study: removing process identity (red dashed) causes the curve to collapse below 0.35 precision at all operating points; removing both state and process (gray dotted) yields a near-flat curve indistinguishable from random, confirming that crossprocess attack events cannot be detected from isolated event features alone.

Table 2: Nghiên cứu cắt bỏ thành phần (phân chia chéo chiến dịch, nhân xuyên tiến trình). Mỗi dòng loại bỏ một hoặc nhiều thành phần khỏi mô hình đầy đủ. Δ là sự thay đổi tương đối của AUPRC sự kiện.

Biến thể	AUPRC Sự kiện	F1 Sự kiện	AUPRC Nút	Δ AUPRC Sự kiện
HyperMamba (Đầy đủ)	0.7586	0.825	0.8939	—
no_cross_entity	0.6968	0.694	0.8592	-8.1%
no_state	0.7234	0.802	0.8366	-4.6%
no_process_identity	0.3407	0.304	0.3466	-55.1%
no_state + no_process	0.1981	0.218	0.2887	-73.9%

trình ngữ nghĩa, mô hình mặc dù giữ lại ngân hàng trạng thái đầy đủ và tập hợp chéo thực thể nhưng không thể tổng quát hóa các mẫu tấn công sang các ID thực thể chưa từng thấy trong Chiến dịch 2. AUPRC sụp đổ từ 0.76 xuống 0.34, và F1 giảm từ 0.825 xuống 0.304. Trong môi trường thử nghiệm này, các tên tệp nhị phân độc hại giống nhau (ví dụ: `pulseaudio`, `gconf-helper`) lặp lại giữa các chiến dịch ngay cả khi UUID thực thể của chúng khác nhau, cung cấp một từ vựng chung cho những tiến trình. Chúng tôi lưu ý rằng sự lặp lại này là một đặc tính của thiết lập thử nghiệm DARPA TC, nơi cùng một công cụ của đội đỏ (red-team) được triển khai trên các lần tấn công; chúng tôi thảo luận về các tác động đối với việc tổng quát hóa trong thế giới thực trong Mục 6 dưới phần Các mối đe dọa đối với tính hợp lệ (Threats to Validity).

Tập hợp chéo thực thể (AUPRC sự kiện giảm -8.1%) là đóng góp cấu trúc chính. Nếu không chia sẻ trạng

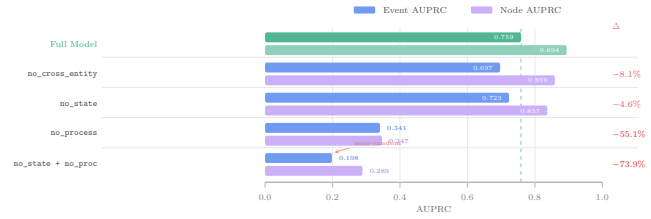


Figure 5: Component ablation on the cross-campaign split with the crossprocess label. The dashed line marks the full model's event AUPRC (0.76). Process identity is the dominant contributor (-55%); removing both state and process identity collapses detection to near-random (0.20).

thái giữa các thực thể trong mỗi siêu cạnh, mô hình không thể truyền vết bẩn (taint) từ một đối tượng bị sửa đổi độc hại sang tiến trình đọc nó. F1 giảm từ 0.825 xuống 0.694 — giảm 15.9% tương đối.

Truyền trạng thái (AUPRC sự kiện giảm -4.6%) đóng góp khả năng phân biệt còn lại cho các sự kiện xuyên tiến trình tinh vi nhất. Mô hình không trạng thái đạt được độ chính xác cao hơn (0.907 so với 0.858) vì chỉ riêng danh tính tiến trình đã cung cấp tín hiệu tin cậy cho các sự kiện xuyên tiến trình rõ ràng nhất. Tuy nhiên, độ bao phủ giảm từ 0.795 xuống 0.715: 8% các sự kiện tấn công bị bỏ lỡ — chính xác là những sự

kiện mà các đặc trưng cục bộ của chúng không thể phân biệt được với hoạt động lành tính. Về mặt vận hành, mô hình không trạng thái sẽ bỏ lỡ khoảng 2,880 sự kiện tấn công mà mô hình đầy đủ phát hiện được.

Loại bỏ kết hợp (AUPRC sự kiện giảm -73.9%) chứng minh rằng loại sự kiện và các đặc trưng liên tục đơn lẻ là không đủ. Ở mức 0.20 AUPRC, hiệu suất phát hiện gần như ngẫu nhiên, xác nhận luận điểm cốt lõi về tính tinh vi (Mục 3.3): các sự kiện tấn công xuyên tiến trình không thể nhận dạng được từ các đặc trưng sự kiện đơn lẻ. Khoảng cách giữa việc loại bỏ kết hợp (0.20) và các nghiên cứu cắt bỏ thành phần đơn lẻ tiết lộ cấu trúc tương tác: trạng thái đóng góp 0.14 AUPRC tăng thêm khi không có danh tính tiến trình (0.20 $\rightarrow$ 0.34) nhưng chỉ đóng góp 0.04 khi có danh tính tiến trình (0.72 $\rightarrow$ 0.76), cho thấy danh tính tiến trình và trạng thái thời gian mang thông tin hành vi trùng lặp một phần, với danh tính tiến trình là cơ chế chuyển giao hiệu quả hơn giữa các chiến dịch.

5.4 Chi phí Tính toán

Một yêu cầu quan trọng đối với bất kỳ NIDS nào là khả năng hoạt động trong thời gian thực mà không bị tụt lại phía sau tốc độ tạo nhật ký kiểm toán của máy chủ. Chúng tôi đánh giá hiệu năng về chi phí tính toán của HyperMamba, tóm tắt kết quả trong Bảng 3.

Table 3: Chi phí Tính toán và Thông lượng

Chỉ số	Giá trị
Tham số (HyperMamba)	1.17M
Bộ nhớ trạng thái (7.1M thực thể $\times d = 256$)	7.3 GB (CPU RAM)
Thông lượng Đào tạo (GPU)	192K sự kiện/giây
Thông lượng Suy luận (GPU)	1.9M sự kiện/giây

Thông lượng. Tập dữ liệu DARPA TC E3 được tạo ra với tốc độ đỉnh điểm khoảng 10,000 sự kiện mỗi giây cho mỗi máy chủ. Như được chỉ ra trong Bảng 3, HyperMamba đạt thông lượng đào tạo là 192K sự kiện/giây và thông lượng suy luận là 1.94M sự kiện/giây trên một GPU duy nhất. Đường ống hoạt động ở mức $19\times$ tốc độ nạp đỉnh điểm trong quá trình đào tạo và $194\times$ trong quá trình suy luận, đảm bảo rằng mô hình sẽ không bao giờ là điểm nghẽn của luồng kiểm toán. Hiệu suất phát hiện ổn định trên các kích thước chunk trong phạm vi 1024–4096 (AUPRC sự kiện thay đổi ít hơn 0.02 trên tập phân chia chéo chiến dịch), cho thấy hiệu suất của mô hình không nhạy cảm với ranh giới cắt ngắn của TBPTT trong phạm vi này.

Bối cảnh thông lượng phương pháp cơ sở. Không giống như suy luận dạng luồng một lần của HyperMamba, các phương pháp cơ sở cấp đồ thị (KAİROS, MAGIC) yêu cầu xây dựng đồ thị ngoại tuyến trước khi quá trình suy luận có thể bắt đầu: đồ thị nguồn gốc đầy đủ cho một cửa sổ phân tích phải được hiện thực hóa,

trích xuất đặc trưng, và mạng thần kinh đồ thị được áp dụng cho cấu trúc hoàn chỉnh. Đường ống hai giai đoạn này (xây dựng + suy luận) tạo ra độ trễ tỷ lệ thuận với kích thước cửa sổ phân tích. Các phương pháp cơ sở cấp nút (FLASH, ThreaTrace) tương tự yêu cầu một ảnh chụp nhanh đồ thị hoàn chỉnh cho mỗi cửa sổ phân tích. Kiến trúc hồi quy của HyperMamba xử lý mỗi sự kiện chính xác một lần, theo thứ tự đến, với tính toán $O(1)$ cho mỗi sự kiện và không có chi phí xây dựng đồ thị — một lợi thế cấu trúc cho việc triển khai thời gian thực.

Bộ nhớ và tham số. Mặc dù hoạt động trên một siêu đồ thị liên tục khổng lồ, HyperMamba cực kỳ hiệu quả về số lượng tham số, chỉ yêu cầu 1.17M tham số có thể đào tạo. Dấu chân bộ nhớ chính là **EntityStateBank**, nằm trong bộ nhớ ghim (pinned memory) của CPU (Mục 4.3). Đối với 7.1 triệu thực thể duy nhất của tập dữ liệu Theia ở mức $d = 256$ (sử dụng dấu phẩy động 32-bit), ngân hàng yêu cầu 7.3 GB RAM hệ thống, để lại GPU VRAM hoàn toàn cho tính toán mô hình. Sự phân tách này cho phép mở rộng quy mô lên các quần thể thực thể vượt quá bộ nhớ GPU mà không cần cơ sở hạ tầng phân tán.

6 Thảo luận

Chúng tôi tuyên bố các giới hạn của nghiên cứu này như các ràng buộc thực tế của đánh giá hiện tại.

Phạm vi tập dữ liệu. Đánh giá trải dài trên hai tập dữ liệu DARPA TC E3 — Theia (Linux) dưới nhãn xuyên tiến trình (crossprocess) và CADETS (FreeBSD) dưới nhãn rộng (broad) — bao gồm các hệ điều hành và vectơ tấn công khác biệt. Khả năng tổng quát hóa sang các khung kiểm toán khác (sysdig, eBPF), môi trường doanh nghiệp với phân phối khối lượng công việc khác biệt đáng kể, hoặc các tập dữ liệu không thuộc DARPA chưa được đánh giá.

Khả năng bao phủ của nhãn. Nhãn xuyên tiến trình nắm bắt các chuỗi tấn công tiến trình con đa giai đoạn nhưng không bao phủ tất cả các loại tấn công. Các cuộc tấn công chỉ dữ liệu (data-only attacks) không bao giờ sinh ra tiến trình mới (ví dụ: các khai thác làm hỏng cấu trúc dữ liệu trong bộ nhớ mà không thực thi các tệp nhị phân mới) nằm ngoài định nghĩa nhãn này và không được giải quyết trong đánh giá hiện tại.

Mở rộng quy mô quần thể thực thể. Ngân hàng **EntityStateBank** phân bổ một vectơ trạng thái có kích thước cố định cho mỗi thực thể duy nhất trong bộ nhớ ghim của CPU. Đối với 7.1 triệu thực thể của Theia ở mức $d = 256$, điều này yêu cầu 7.3 GB RAM hệ thống. Các triển khai với quần thể thực thể lớn hơn đáng kể so với tập dữ liệu Theia có thể yêu cầu các chiến lược xấp xỉ như phân cụm thực thể hoặc nén trạng thái; tác động đối với hiệu suất phát hiện chưa được đánh giá

trong nghiên cứu này.

Tính bền vững trước kẻ tấn công. Đánh giá chính giả định một kẻ tấn công hợp đen (Mục 3.1). Một đối thủ hợp trắng hiểu cơ chế truyền trạng thái SSM có khả năng tạo ra các chiến lược lẩn tránh — ví dụ, tiếm các chuỗi sự kiện lành tính để pha loãng trạng thái tích lũy của một thực thể bị xâm phạm trước khi thực hiện các hoạt động tấn công. Tính bền vững của HyperMamba chống lại các cuộc tấn công pha loãng trạng thái có mục tiêu như vậy chưa được đánh giá.

Các mối đe dọa đối với tính hợp lệ. Vai trò vượt trội của danh tính tiến trình trong nghiên cứu cắt bỏ (−55%) cần được diễn giải cẩn thận. Môi trường thử nghiệm DARPA TC E3 tái sử dụng cùng các công cụ của đội đỏ trên các chiến dịch, vì vậy cùng các tên tiến trình (*pulseaudio*, *gconf-helper*) xuất hiện trong cả chiến dịch đào tạo và kiểm tra. Trong triển khai thực tế, kẻ tấn công có thể sử dụng các tên tệp nhị phân tùy ý hoặc chưa từng thấy trước đây. Cơ chế danh tính tiến trình khi đó sẽ hoạt động như một phân phối tiên nghiệm học được trên các công cụ đã biết thay vì một tín hiệu phát hiện vạn năng; trong những thiết lập như vậy, trạng thái liên tục và tập hợp chéo thực thể — vốn độc lập với tên tiến trình — có khả năng sẽ trở nên quan trọng hơn tương đối. Đánh giá giả thuyết này (ví dụ: bằng cách ẩn danh các tên tiến trình để không có chuỗi ký tự nào trùng lặp giữa đào tạo và kiểm tra) là một ưu tiên cho nghiên cứu tương lai.

Ngoài ra, tất cả các kết quả được báo cáo từ một seed ngẫu nhiên duy nhất (seed 42). Mặc dù các tác động cắt bỏ có quy mô lớn (−55%, −74%) ít có khả năng phụ thuộc vào seed, các tác động nhỏ hơn (−4.6%, −8.1%) có thể trùng lặp với phương sai của seed. Đánh giá đa seed với các khoảng tin cậy được báo cáo là cần thiết để xác nhận thứ tự của phân cấp cắt bỏ.

Các cuộc tấn công dựa trên tải trọng. HyperMamba hoạt động độc quyền trên siêu dữ liệu kiểm toán có cấu trúc (loại sự kiện, số byte, dấu thời gian). Nó không kiểm tra tải trọng gói tin mạng hoặc nội dung tệp. Các cuộc tấn công chỉ có thể phân biệt được bằng nội dung tải trọng (ví dụ: tấn công SQL injection trong thân yêu cầu HTTP) nằm ngoài phạm vi của hệ thống này.

Phạm vi máy chủ đơn. Đánh giá hiện tại xem xét các luồng kiểm toán từ một máy chủ được giám sát duy nhất. Việc di chuyển ngang nhiều máy chủ (multi-host lateral movement) — nơi kẻ tấn công chuyển tiếp giữa các máy trong môi trường doanh nghiệp phân tán — yêu cầu liên kết trạng thái chéo máy chủ, điều này chưa được giải quyết trong nghiên cứu này.

7 Nghiên cứu Liên quan

Chúng tôi định vị HyperMamba so với các nghiên cứu trước đây trên ba trục: độ phân giải phát hiện, cơ chế

Table 4: Định vị HyperMamba so với các NIDS dựa trên nguồn gốc trên ba trục.

Hệ thống	Độ phân giải	Trạng thái	Giám sát
KAIROS	Graph	Ảnh chụp nhanh (Snapshot)	Tự giám sát
MAGIC	Graph	Ảnh chụp nhanh (Snapshot)	Tự giám sát
FLASH	Node	Ảnh chụp nhanh (Snapshot)	Tự giám sát
ThreaTrace	Node	Ảnh chụp nhanh (Snapshot)	Có giám sát
HyperMamba	Event	Liên tục (Persistent)	Có giám sát

trạng thái thời gian, và chế độ giám sát. Bảng 4 tóm tắt bối cảnh này.

NIDS dựa trên nguồn gốc. KAIROS [3] xây dựng các đồ thị nguồn gốc thời gian và gán điểm dị thường ở cấp độ đồ thị thông qua học dựa trên tái cấu trúc. MAGIC [7] tương tự hoạt động ở cấp độ đồ thị, phát hiện các cuộc tấn công thông qua lỗi tái cấu trúc đồ thị con cấu trúc. FLASH [8] cung cấp các cảnh báo cấp nút bằng cách xác định các tiến trình hoặc tệp dị thường trong đồ thị nguồn gốc. ThreaTrace [9] phân loại các thực thể riêng lẻ (nút) sử dụng mạng thần kinh đồ thị. Cả bốn hệ thống đều mô hình hóa nguồn gốc dưới dạng đồ thị cặp đôi, loại bỏ cấu trúc đa đối tượng nguyên bản của các sự kiện CDM. Quan trọng hơn, không hệ thống nào tạo ra phân loại ở cấp độ sự kiện — mức độ chi tiết nhỏ nhất cần thiết cho quy kết pháp y của từng bước tấn công riêng lẻ.

Cơ chế trạng thái thời gian. Các hệ thống trên hoạt động trên các ảnh chụp nhanh đồ thị tĩnh hoặc theo cửa sổ, xây dựng một đồ thị mới cho mỗi cửa sổ phân tích và loại bỏ trạng thái thực thể giữa các cửa sổ. HyperMamba duy trì các vectơ trạng thái thực thể liên tục trên toàn bộ luồng sự kiện thông qua một SSM Chọn lọc, cho phép mô hình tích lũy bối cảnh hành vi trên các khoảng thời gian dài tùy ý. Thử nghiệm cắt bỏ trong Mục 5.3 chứng minh rằng trạng thái liên tục này, kết hợp với danh tính tiến trình ngữ nghĩa, là cần thiết đồng thời cho việc phát hiện — loại bỏ cả hai thành phần làm sụp đổ AUPRC xuống 0.20.

Sự đánh đổi về giám sát. MAGIC và KAIROS là tự giám sát: chúng đào tạo độc quyền trên dữ liệu lành tính và gán cờ các sai lệch thống kê, tránh sự cần thiết của dữ liệu đào tạo được gán nhãn tấn công. HyperMamba là có giám sát: nó yêu cầu các nhãn tấn công xuyên tiến trình trong quá trình đào tạo. Đây là một lựa chọn thiết kế có chủ ý. Các hệ thống dựa trên phát hiện dị thường hoạt động ở cấp độ đồ thị hoặc cửa sổ mà không có quy kết cấp sự kiện vì mục tiêu tái cấu trúc của chúng không liên kết các sự kiện riêng lẻ với nhãn tấn công. Công thức giám sát của chúng tôi yêu cầu nhãn xuyên tiến trình nhưng cho phép độ chính xác phát hiện cấp sự kiện mà các phương pháp tự giám sát không thể cung cấp. Việc dữ liệu xuyên tiến trình có nhãn có sẵn trong thực tế hay không phụ thuộc vào bối cảnh triển khai; chúng tôi thảo luận về hạn chế này trong Mục 6.

Phát hiện xâm nhập dựa trên siêu đồ thị. Một số hệ thống đã khám phá các biểu diễn siêu đồ thị cho phát hiện xâm nhập, mặc dù trong các lĩnh vực khác nhau và với các mục tiêu khác nhau. LOHA [2] sử dụng phát hiện dị thường dựa trên tái cấu trúc trên các siêu đồ thị được xây dựng từ các luồng mạng. HyperGAT [4] áp dụng sự chú ý siêu đồ thị cho khai thác cộng đồng tĩnh. HRNN [10] xây dựng các siêu cạnh thông qua KNN trên các vectơ đặc trưng luồng mạng. Không có hệ thống nào trong số này sử dụng các siêu cạnh xác định, cấp sự kiện được rút ra từ mô hình dữ liệu kiểm toán, và không hệ thống nào duy trì trạng thái thực thể liên tục trên luồng.

Mô hình Không gian Trạng thái. SSM Chọn lọc được giới thiệu bởi Mamba [5] để mô hình hóa chuỗi hiệu quả trong xử lý ngôn ngữ tự nhiên (NLP) và thị giác máy tính, xây dựng trên nền tảng SSM cấu trúc của S4 [6] và S6 [5]. HyperMamba điều chỉnh SSM Chọn lọc sang một thiết lập cơ bản khác biệt: thay vì xử lý một chuỗi mã thông báo (token) duy nhất, SSM hoạt động trên các vectơ trạng thái cho từng thực thể bên trong một siêu đồ thị thời gian, với bước rời rạc hóa Δ được điều kiện hóa theo thời gian đã trôi qua đặc thù của thực thể (Mục 4.5).

8 Kết luận

Chúng tôi đã trình bày HyperMamba, một NIDS dựa trên nguồn gốc giúp phát hiện các sự kiện tấn công xuyên tiến trình riêng lẻ trong các luồng kiểm toán liên tục bằng cách duy trì trạng thái thực thể liên tục thông qua Mô hình Không gian Trạng thái Chọn lọc. Nghiên cứu cắt bỏ tiết lộ một phân cấp rõ ràng về đóng góp của các thành phần: danh tính tiến trình ngữ nghĩa là cơ chế vượt trội cho việc tổng quát hóa chéo chiến dịch (giảm -55% khi bị loại bỏ), tập hợp siêu cạnh chéo thực thể mang lại đóng góp cấu trúc chính (-8.1%), và trạng thái thời gian tích lũy bổ sung khả năng phân biệt còn lại cho các sự kiện tĩnh vi nhất (-4.6%). Việc loại bỏ cả trạng thái và danh tính tiến trình làm sụp đổ hiệu suất phát hiện về gần mức ngẫu nhiên (0.20 AUPRC), xác nhận rằng các sự kiện tấn công xuyên tiến trình không thể nhận dạng được từ các đặc trưng sự kiện đơn lẻ. Trên các tập dữ liệu DARPA TC E3, HyperMamba đạt được AUPRC cấp sự kiện là 0.76 trên Theia (nhân xuyên tiến trình, chéo chiến dịch) và 0.97 trên CADETS (nhân rộng, chéo chiến dịch), với $FPR \leq 0.0011$, trong khi xử lý các sự kiện với tốc độ gấp $194\times$ tốc độ thu thập dữ liệu kiểm toán đỉnh điểm khi suy luận.

Một số hướng đi xứng đáng được nghiên cứu sâu hơn. Đào tạo tiền đề tự giám sát (self-supervised pretraining) trên các luồng kiểm toán không được gắn nhãn — sử dụng dự đoán sự kiện bị che khuất hoặc các mục tiêu trạng thái thực thể tương phản — có thể giảm bớt sự

phụ thuộc vào các nhãn tấn công xuyên tiến trình, vốn rất tốn kém để thu thập ngoài các cuộc thử nghiệm có kiểm soát. Mở rộng ngân hàng trạng thái thực thể để liên kết các vectơ trạng thái trên nhiều máy chủ được giám sát sẽ cho phép phát hiện các chiến dịch di chuyển ngang, nơi tín hiệu phân biệt trải dài qua ranh giới các máy chủ. Các kỹ thuật nén trạng thái (ví dụ: các phép chiếu hạng thấp hoặc phân cụm thực thể) có thể giảm dấu chân bộ nhớ cho các triển khai với quần thể thực thể lớn hơn nhiều bậc so với các tập dữ liệu chuẩn DARPA TC. Đánh giá hiện tại bị giới hạn trong họ tập dữ liệu DARPA TC E3; khả năng tổng quát hóa sang các khung kiểm toán và phân loại tấn công khác vẫn cần được xác nhận.

Xem xét về Đạo đức

Tất cả các thử nghiệm trong bài báo này sử dụng các tập dữ liệu DARPA Transparent Computing Engagement 3 có sẵn công khai, được thu thập trong môi trường thử nghiệm có kiểm soát không có người dùng thực hoặc dữ liệu riêng tư. Không có đối tượng con người nào tham gia vào bất kỳ phần nào của nghiên cứu này. Đánh giá của IRB không được yêu cầu. Mô hình mối đe dọa và mô tả cuộc tấn công dựa trên các hoạt động của đội đỏ đã được tài liệu hóa trong chương trình DARPA TC và không tiết lộ các lỗ hổng hoặc kỹ thuật tấn công mới.

Khoa học Mở và Khả dụng của Học liệu

Để hỗ trợ khả năng tái lập, mã nguồn của HyperMamba — bao gồm đường ống tiền xử lý, triển khai mô hình, tập lệnh đào tạo, và bộ khung đánh giá — sẽ được phát hành dưới dạng một kho lưu trữ nguồn mở khi bài báo được công bố. Các tập dữ liệu DARPA TC E3 được sử dụng trong đánh giá này có sẵn công khai. Chúng tôi dự kiến tham gia vào quy trình Đánh giá Học liệu (Artifact Evaluation) của USENIX.

References

- [1] Daniel Arp, Erwin Quiring, Feargus Pendlebury, Alexander Warnecke, Fabio Pierazzi, Christian Wressnegger, Lorenzo Cavallaro, and Konrad Rieck. Dos and don'ts of machine learning in computer security. In 31st USENIX Security Symposium (USENIX Security 22), pages 3971–3988, 2022.
- [2] Guanwen Chen, Lizhao Wu, Hui Lin, and Zhaobin Zhou. LOHA: Hypergraph masked autoencoder for advanced persistent threat detection. In 2025

- IEEE International Conference on High Performance Computing and Communications (HPCC). IEEE, 2025.
- [3] Zijun Cheng, Qiujian Lv, Jinyuan Liang, Yang Wang, Degang Sun, Thomas Pasquier, and Xueyuan Han. Kairos: Practical intrusion detection and investigation using whole-system provenance. In 2024 IEEE Symposium on Security and Privacy (SP), pages 3533–3551, 2024.
 - [4] Kaize Ding, Jianling Wang, Jundong Li, Dingcheng Li, and Huan Liu. Be more with less: Hypergraph attention networks for inductive text classification. In Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), pages 4927–4936. Association for Computational Linguistics, 2020.
 - [5] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. In First Conference on Language Modeling, 2024.
 - [6] Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces. In The Tenth International Conference on Learning Representations, 2022.
 - [7] Zian Jia, Yun Xiong, Yuhong Nan, Yao Zhang, Jinjing Zhao, and Mi Wen. MAGIC: Detecting advanced persistent threats via masked graph representation learning. In 33rd USENIX Security Symposium (USENIX Security 24), pages 5197–5214, 2024.
 - [8] Mati Ur Rehman, Hadi Ahmadi, and Wajih Ul Hassan. Flash: A comprehensive approach to intrusion detection via provenance graph representation learning. In 45th IEEE Symposium on Security and Privacy (SP), pages 3552–3570. IEEE, 2024.
 - [9] Su Wang, Zhiliang Wang, Tao Zhou, Hongbin Sun, Xia Yin, Dongqi Han, Han Zhang, Xingang Shi, and Jiahai Yang. Threatrace: Detecting and tracing host-based threats in node level through provenance graph learning. IEEE Transactions on Information Forensics and Security, 17:3972–3987, 2022.
 - [10] Zhe Yang, Zitong Ma, Wenbo Zhao, Lingzhi Li, and Fei Gu. HRNN: Hypergraph recurrent neural network for network intrusion detection. Journal of Grid Computing, 22(2):52, 2024.